

RubricEM: Meta-RL with Rubric-guided Policy Decomposition beyond Verifiable Rewards

Gaotang Li¹, Bhavana Dalvi Mishra², Zifeng Wang², Jun Yan², Yanfei Chen², Chun-Liang Li², Long T. Le², Rujun Han², George Lee², Hanghang Tong¹, Chen-Yu Lee² and Tomas Pfister²

¹University of Illinois Urbana-Champaign, ²Google Cloud AI Research

Training deep research agents—systems that plan, search, evaluate evidence, and synthesize long-form reports—pushes reinforcement learning beyond the regime of verifiable rewards. Their outputs lack ground-truth answers, their trajectories span many tool-augmented decisions, and standard post-training offers little mechanism for turning past attempts into reusable experience. In this work, we argue that rubrics should serve not merely as final-answer evaluators, but as the shared interface that structures policy execution, judge feedback, and agent memory. Based on this view, we introduce RUBRICEM, a rubric-guided reinforcement learning framework that combines stagewise policy decomposition with reflection-based meta-policy training. RUBRICEM first makes research trajectories stage-aware by conditioning planning, evidence gathering, review, and synthesis on self-generated rubrics. It then assigns credit with Stage-Structured GRPO, which uses stagewise rubric judgments to provide denser semantic feedback for long-horizon optimization. In parallel, RUBRICEM trains a shared-backbone reflection meta-policy that distills judged trajectories into reusable rubric-grounded guidance for future attempts. The resulting RUBRICEM-8B achieves strong performance across four representative long-form research benchmarks, outperforming comparable open models and approaching proprietary deep-research systems. Beyond final performance, we perform thorough analyses to understand the key ingredients of RUBRICEM.

1. Introduction

Deep research agents answer complex information-seeking questions by autonomously planning, searching, evaluating evidence, and synthesizing long-form reports. Yet how to train this capability remains unclear: proprietary systems such as Gemini and OpenAI’s deep research (Google DeepMind, 2025; OpenAI, 2025) reveal little about their methodology, while most existing efforts rely on verifiable search proxies (Jin et al., 2025; Nguyen et al., 2025; Song et al., 2025) or high-quality imitation data (Moonshot AI, 2025; Perplexity AI, 2025; Tongyi DeepResearch Team et al., 2025). End-to-end RL for long-form research is difficult because outputs lack ground-truth verification, judge feedback is coarse and delayed over long tool-augmented rollouts, and conventional post-training mostly converts judged attempts into parametric updates without producing explicit reusable guidance. This raises the central question of this work:



How can reinforcement learning train deep research agents beyond verifiable rewards, while enabling long-horizon credit assignment and learning from experience?

Rubrics offer a natural handle for open-ended tasks whose quality cannot be verified by exact answers (Chen et al., 2025b; Gunjal et al., 2025; Shao et al., 2025). Prior work mainly uses them as judge-side criteria for assigning rewards to final responses. Our key perspective is that rubrics should instead serve as a shared interface throughout reinforcement learning. The same criteria that define success can guide the agent’s planning and search, support process-level judgment over intermediate decisions, and be distilled into reusable reflections for learning from experience. Based on this view, we propose RUBRICEM, a rubric-guided reinforcement learning framework that combines stagewise

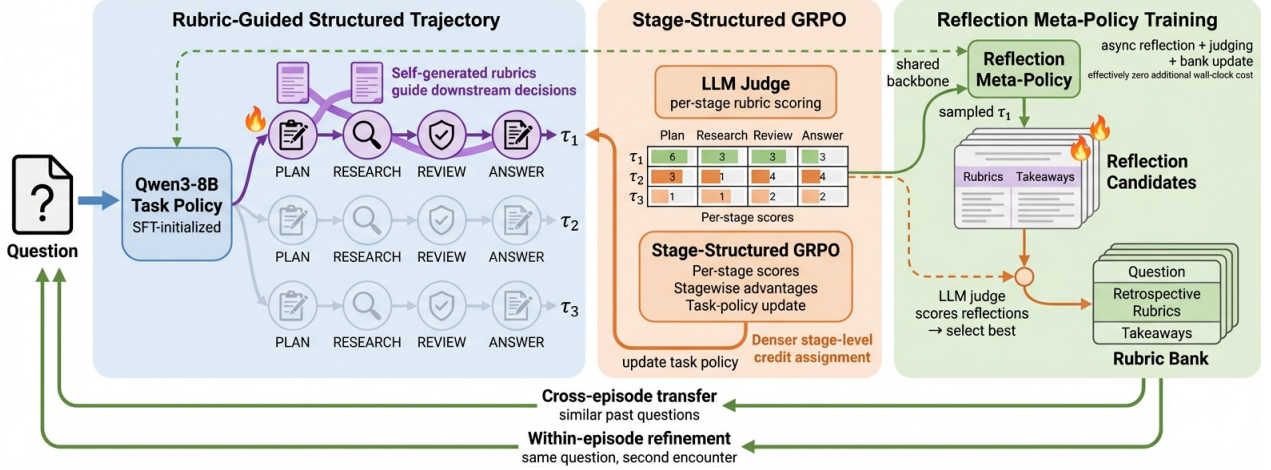


Figure 1 | RUBRICEM first instills a rubric-guided structured scaffold into the task policy, so that each trajectory follows stage-structured reasoning under self-generated rubrics. During RL, we propose Stage-Structured GRPO to provide finer-grained, denser credit assignment. In parallel, a shared-backbone reflection meta-policy is jointly trained to generate rubric-grounded reflections, which are stored in a rubric bank to support both cross-episode transfer and within-episode refinement. Together, rubrics serve as the shared interface across the agent, the judge, and their evolutions.

policy decomposition with reflection-based meta-policy evolution. The name RUBRICEM reflects an **Expectation–Maximization (EM)-inspired estimate–maximize view** (Dempster et al., 1977) (beyond supervised settings): the latent structure of an open-ended research task—what matters, where credit belongs, and what should be remembered—is estimated through rubrics, which condition policy reasoning, judge scoring, and memory evolution. Training then maximizes the task policy and reflection meta-policy under these rubric-conditioned estimates.

RUBRICEM first realizes rubric-guided policy decomposition through a **rubric-guided reasoning scaffold**. During planning, the agent generates task-specific rubrics and carries them through four stages: planning, research, review, and answer synthesis. This converts a flat long-horizon rollout into rubric-conditioned decision stages, where each stage defines both a distinct decision mode and a natural unit for optimization. The scaffold also makes rubrics operational across the training loop: they guide search and synthesis, serve as on-policy references for the judge, and produce structured traces that can be distilled into reusable reflections.

Building on this decomposition, RUBRICEM assigns credit with **Stage-Structured GRPO** (SS-GRPO). Rather than broadcast a single terminal score to all tokens, SS-GRPO scores Plan, Research, Review, and Answer with stage-specific rubrics. The judge maintains an evolving rubric buffer for each stage, extending prior evolving-rubric evaluation (Shao et al., 2025) from final-answer judging to process-level feedback. These stagewise scores define denser returns that combine local stage quality with downstream impact, giving GRPO finer-grained credit signals while remaining critic-free.

Finally, RUBRICEM makes experience reuse an explicit RL objective through **Reflection Meta-Policy** training. The task policy and reflection meta-policy share one backbone: after a task rollout is judged, the backbone samples rubric-grounded reflection candidates conditioned only on the query and raw trajectory, while a separate judge scores these candidates using the task-rollout judgments. The reflection scores provide auxiliary RL rewards on reflection tokens, updating the shared parameters; the highest-scored accepted reflection is also written into an agent rubric bank as natural-language memory. The bank conditions future rollouts in two modes: within-episode refinement retrieves the previous reflection for the same query, while cross-episode transfer retrieves reflections from related

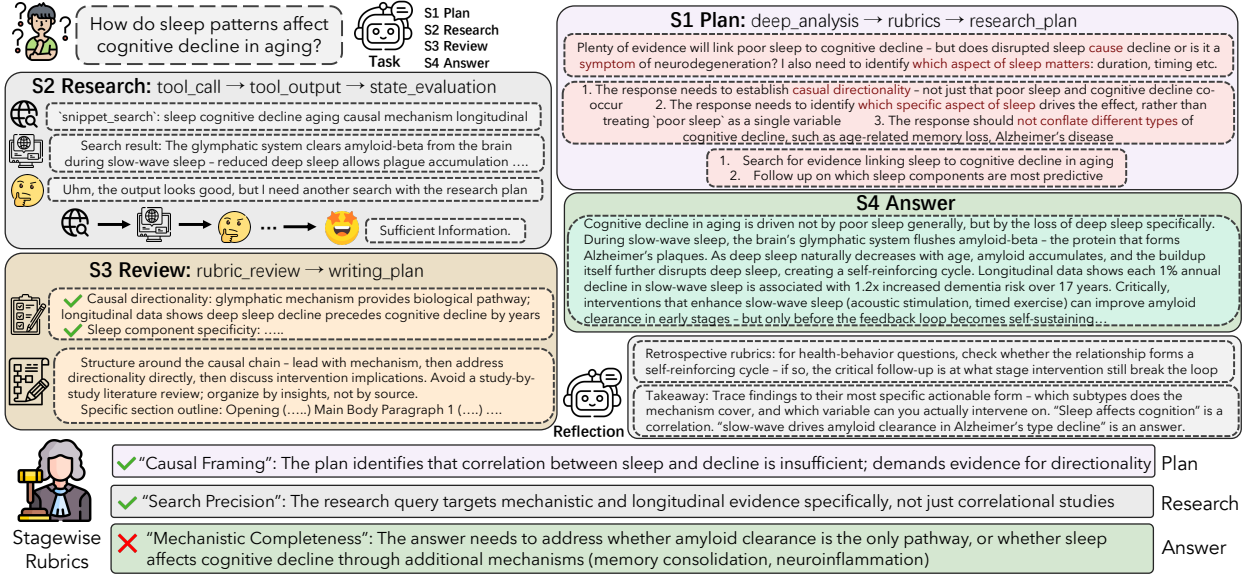


Figure 2 | Example of the rubric-guided stage-structured search agent trajectory, meta-policy reflection, and stagewise judge rubrics during a single RL step.

questions. Thus, each reflection updates the agent both parametrically and textually. We designed an efficient asynchronous reflection branch to train this meta-policy alongside task-policy RL without adding a sequential bottleneck, a notable problem in prior meta-RL literature (Jiang et al., 2026).

Together, these components yield RUBRICEM-8B, an 8B deep research agent trained with 1400 RL steps. Fig. 1 gives an overview of the framework, and Fig. 2 illustrates a concrete example. Across four representative long-form research benchmarks, RUBRICEM-8B achieves state-of-the-art performance among comparable open models, improves over strong prior RL systems with fewer training steps, and approaches proprietary deep-research systems such as Gemini and OpenAI Deep Research. Beyond final scores, we conduct extensive ablations and analyses, including multiple 600-step RL ablations, scaffold comparisons, inference scaling, and out-of-domain short-form transfer. These results support a broader recipe for long-horizon RL beyond verifiable rewards: expose task structure, assign credit to that structure, and convert judged attempts into reusable experience.

2. Related Work

The Post-training recipes of deep research agents. Most existing open-source training efforts for deep research focus on short-answer tasks with verifiable rewards (Chen et al., 2025a; Han et al., 2025; Jiang et al., 2025; Jin et al., 2025; Song et al., 2025; Zhao et al., 2025). Meanwhile, proprietary systems mainly report scaling high-quality imitation data and training on verifiable short-form settings (Google, 2025; Moonshot AI, 2025; OpenAI, 2025; Perplexity AI, 2025). Our work takes an orthogonal direction: making reinforcement learning effective for open-ended, long-form deep research. The closest work to ours is DR Tulu (Shao et al., 2025), which studies end-to-end RL for deep research beyond verifiable rewards. We build on this foundation by introducing fine-grained credit assignment and jointly trained meta-policy evolution, yielding denser learning signals and reusable guidance during the challenging long-horizon RL process.

Credit assignment and meta-RL with language models. Recent work on agentic reinforcement learning has increasingly emphasized the need for finer-grained credit assignment (DeepSeek-AI,

2026; Mousavi-Hosseini and Erdogdu, 2026; Qian et al., 2025; Xi et al., 2026; Zhang, 2026). However, most of these methods operate in verifiable settings, where trajectories can be decomposed into subgoals with reliable process-level supervision. A related line of work trains meta-policies during reinforcement learning, often referred to as Meta-RL (Jiang et al., 2026; Yang et al., 2026a). While promising, these methods are typically evaluated on verifiable or synthetic tasks and often introduce explicit dependencies across rollouts, leading to substantial training overhead. In contrast, our work targets open-ended real-world deep research tasks, where neither intermediate progress nor final answers admit simple automatic verification. We improve meta-policy training efficiency by removing cross-rollout dependencies and designing an efficient reflection-training infrastructure.

3. RUBRICEM

3.1. Preliminaries and notations

We study deep research agents for complex information-seeking queries. Given a query $q \sim \mathcal{D}$, an agent interacts with a tool environment $\mathcal{T}$ and produces a trajectory $\tau = (a_1, o_1, \dots, a_T, o_T)$, where a_t denotes the agent emission at turn t , which may be either a textual segment or a structured tool call, and o_t is the resulting tool output (with $o_t = \emptyset$ when no tool is invoked). We consider a language-model-based agent that autoregressively samples the next step $a_t \sim \pi_\theta(a_t | h_t)$, $h_t = (q, a_{<t}, o_{<t})$, and eventually produces a final long-form answer grounded in retrieved evidence.

3.2. Structured Reasoning Scaffold

A central design choice in RUBRICEM is to impose explicit stage structure on agent trajectories. A *stage* refers to a semantically defined segment of the trajectory that serves a distinct decision role, such as planning, evidence gathering, self-evaluation, or final synthesis. In long-horizon research tasks, these stages provide a stable high-level organization over otherwise noisy token-level generation. When these decision modes are collapsed into a flat autoregressive process, the trajectory lacks such stage-level organization. The policy must therefore infer its current decision mode from local context alone, which can lead to inefficient exploration and compounding errors over long horizons (Feng et al., 2026; Xu et al., 2025). We formalize the value of explicit stage information as follows. Let h denote a random decision point along a trajectory induced by the current policy, let $c = \phi(h)$ denote a compressed state representation, let z denote the current stage label, and let $U(h, a)$ denote the expected downstream value of taking action a at history h and then continuing the rollout.

Theorem 1 (Value of stage information). *Under mild assumptions in Assump. 1, define*

$$V_{\text{flat}} := \mathbb{E} \left[\max_{a \in \mathcal{A}} \mathbb{E}[U(h, a) | c] \right], \quad V_{\text{stage}} := \mathbb{E} \left[\max_{a \in \mathcal{A}} \mathbb{E}[U(h, a) | c, z] \right].$$

If there exists a measurable set C_0 with positive probability and two task-relevant stages such that for every $c \in C_0$, $p(z | c) > 0$, $p(z' | c) > 0$, and that $\arg \max_{a \in \mathcal{A}} \mathbb{E}[U(h, a) | c, z] \cap \arg \max_{a \in \mathcal{A}} \mathbb{E}[U(h, a) | c, z'] = \emptyset$. Then

$$V_{\text{stage}} > V_{\text{flat}}.$$

Theorem 1 identifies when explicit stage information is beneficial. In long-horizon research trajectories, the same local context may call for different actions across planning, searching, reviewing, and final synthesis. When these stage-specific optimal actions disagree, a flat policy acts under an aliased context, whereas a stage-aware policy can condition on the current decision mode. This yields a strict value improvement on any positive-probability set where such aliasing occurs. We

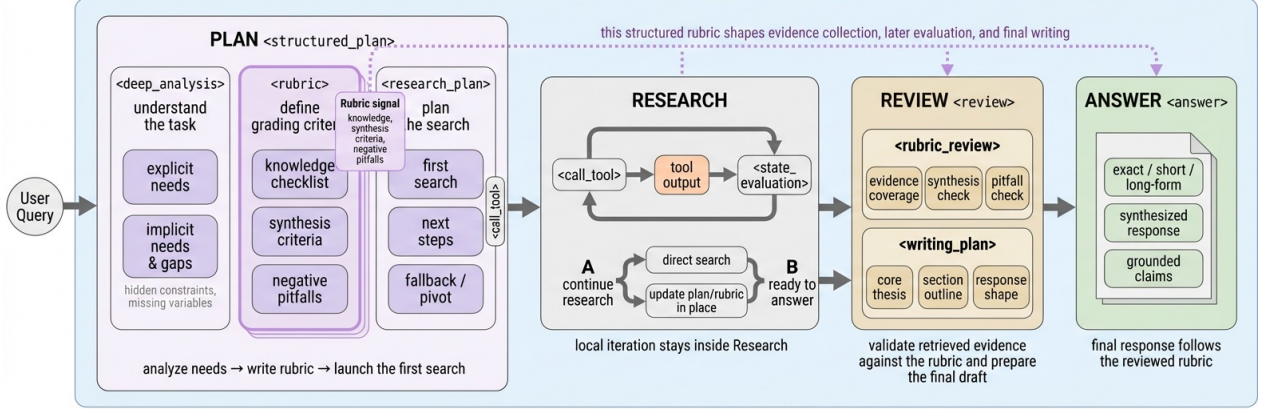


Figure 3 | Rubric-guided structured reasoning scaffold in RUBRICEM. The agent follows a high-level workflow of Plan, Research, Review, and Answer, each with a lightweight internal schema. Rubrics are generated in Plan and then guide evidence collection, evaluation, and final writing. During Research, the agent iterates between tool use and state evaluation; once evidence is sufficient, it enters Review and then produces the final grounded Answer. The full description is in Appen. B.1

therefore make stage structure explicit rather than implicit in a flat trajectory. The proof is deferred to Appen. E.1.

Specific stage instantiation. We instantiate this idea with four rubric-guided stages:

PLAN → RESEARCH → REVIEW → ANSWER. Each stage is marked by a stage-level XML tag with a lightweight internal schema. The outer scaffold is sequential, while RESEARCH allows local iteration and in-place plan revision. The overview is in Fig. 3 and detailed below:

PLAN. Within `<structured_plan>`, the agent analyzes the user’s explicit and implicit needs in `<analysis>`, translates them into `<rubrics>`, and then proposes a concrete `<research_plan>`. The rubrics specify (i) a knowledge checklist of information to gather, (ii) analytical criteria for the final write-up, and (iii) negative constraints on what the answer should avoid.

RESEARCH. The agent iteratively issues `<call_tool>` actions. After each tool response, the agent performs a `<state_evaluation>` step, which compares the accumulated evidence against the plan and rubrics, decides whether further search is needed, and optionally revises the PLAN in place.

REVIEW. Within `<review>`, the agent maps collected evidence back to the rubrics through `<rubrics_review>` and prepares a writing plan, including the main thesis and section outline.

ANSWER. Within `<answer>`, the agent synthesizes the final long-form response with citation support.

Importantly, rubrics are not merely evaluation artifacts in RUBRICEM: they are generated in PLAN, can be revised during RESEARCH, and guide subsequent stages throughout the trajectory. This instantiation is motivated by three considerations. First, it is task-aligned: deep research naturally involves planning, evidence acquisition, self-evaluation, and synthesis, so the four stages reflect the task rather than an arbitrary template. Second, explicit criteria give the policy a stable target for planning, self-checking, and feedback, echoing rubric-based learning and explicit-principle approaches (Bai et al., 2022; Panadero, 2017; Tai et al., 2018). Third, because self-generated rubrics vary across rollouts of the same query, they provide per-rollout references that help the judge discover more aligned and discriminative stagewise criteria. We validate the effectiveness of the scaffold and its importance for RL in Sec. 5.2.

Finally, the scaffold directly enables our later RL design: stage boundaries define the units for SS-GRPO credit assignment, and rubric-conditioned traces define the memory format for the rubric bank (Sec. 3.4). We therefore view the scaffold not as a formatting warm-up, but as an SFT-induced structural prior that prepares the policy for effective RL (Zhang et al., 2026).

SFT distillation. To instantiate the scaffold in the policy, we perform teacher–student distillation from Gemini-3.1-Pro. For each query, the teacher is prompted to produce a stage-structured trajectory that follows the XML schema above. Because raw teacher traces do not always obey the target scaffold, we apply rejection sampling to discard outputs that violate stage boundaries, tool-calling syntax, citation format, or grounding constraints. The resulting SFT corpus teaches Qwen3-8B not only tool use and evidence citation, but also the stage discipline and rubric conditioning required by our later RL design. We defer details of the data-generation and filtering pipeline to Append. B.2.

3.3. Stage-Structured GRPO

Building on the structured scaffold above, we propose **Stage-Structured GRPO (SS-GRPO)** for finer-grained credit assignment in deep research. Prior work on process supervision and long-horizon agent RL suggests that denser process-level rewards can substantially improve credit assignment (Qian et al., 2025; Tan et al., 2026; Wang et al., 2026; Wu et al., 2026; Yang et al., 2026b). However, in open-ended deep research, we do not have oracle intermediate rewards: the quality of planning, search, review, and synthesis is semantic, task-dependent, and difficult to verify automatically. SS-GRPO therefore uses the explicit stage boundaries from Sec. 3.2 together with rubric-guided judging to construct stage-level learning signals, as illustrated in Fig. 4.

Stagewise scores and returns. Given a query q , we sample n rollouts $\{\tau_i\}_{i=1}^n \sim \pi_\theta(\cdot | q)$ and partition each into K semantic stages; in our instantiation, $K = 4$ corresponding to Plan, Research, Review, and Answer. Let $\mathcal{B}_{i,k}$ be the tokens in stage k of rollout τ_i , and let $R_{i,k} \in [0, 1]$ be the LLM-judge score under the corresponding stage rubric. Rather than assign the same final score to all tokens, SS-GRPO uses a causal stage-dependence matrix $\Lambda = (\lambda_{k,j})$, with $\lambda_{k,j} = 0$ for $j < k$ and $\lambda_{k,k} = 1$, and defines $G_{i,k}^\Lambda = \sum_{j=k}^K \lambda_{k,j} R_{i,j}$. Thus each stage keeps its own score while receiving credit from downstream stages it enables. Terminal reward broadcast is considered a special case.

When stage returns help. The benefit of stage returns depends on a simple trade-off: intermediate judging recovers process information omitted by terminal-only rewards, but also introduces judge noise. Appendix E.2, Thm. 3 formalizes this intuition: stage-weighted credit improves the gradient approximation when the recovered intermediate signal outweighs cumulative judge misalignment. Thus SS-GRPO needs no oracle process reward, only sufficiently aligned stagewise judging with bounded noise. This motivates the stagewise evolving-rubric judge below.

Stagewise evolving-rubric judge. As shown in the top panel of Fig. 4, the judge contrasts multiple rollouts for the same query and proposes discriminative rubrics for each stage. The judge maintains a separate rubric buffer for Plan, Research, Review, and Answer, reuses previous high-discrimination rubrics, and removes items that no longer separate trajectory quality. Because the policy trajectories are themselves rubric-guided, the judge can also use trajectory-generated rubrics as references when constructing new judge rubrics, while still scoring trajectories against the judge-side rubric buffer rather than blindly rewarding a rollout’s own self-rubric. This makes the intermediate rewards both stage-local and adaptive to the current policy distribution. Further details are deferred to Append. C

Stagewise normalization and objective. We instantiate SS-GRPO as a critic-free stagewise

variant of GRPO by normalizing returns separately within each stage across the rollout group:

$$A_{i,k} = \frac{G_{i,k}^\Lambda - \frac{1}{n} \sum_{i'=1}^n G_{i',k}^\Lambda}{\text{Std}_{i'}[G_{i',k}^\Lambda] + \epsilon}.$$

All tokens in the same stage block $\mathcal{B}_{i,k}$ share the advantage $A_{i,k}$. The resulting objective is

$$\mathcal{L}_{\text{SS-GRPO}} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K \sum_{t \in \mathcal{B}_{i,k}} \min\left(\rho_{i,t} A_{i,k}, \text{clip}(\rho_{i,t}, 1 - \eta, 1 + \eta) A_{i,k}\right) + \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}), \quad (1)$$

where $\rho_{i,t} = \frac{\pi_\theta(a_{i,t}|h_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t}|h_{i,t})}$. We keep the estimator critic-free because stage supervision is judge-defined, evolving during training, and collected from expensive long-horizon tool-augmented rollouts; adding a learned stage-conditioned critic would introduce substantial additional complexity.

3.4. Meta-Policy Training with Reinforcement Learning

Beyond single-trajectory optimization, RUBRICEM makes experience reuse part of RL. A shared backbone serves as both the task policy and a reflection meta-policy: rubric-guided task rollouts provide judged experience, and the reflection policy is trained with LLM-judge rewards to produce reusable natural-language guidance. Accepted reflections enter an *agent rubric bank* for future retrieval, giving the agent both parametric RL updates and textual memory updates. This retains the meta-RL goal of improving future rollouts from past experience, while our asynchronous design avoids a sequential rollout-reflection-update bottleneck.

Joint training of the reflection meta-policy. After task-policy rollouts are judged, we sample a query-trajectory pair and prompt the shared backbone to generate multiple reflection candidates, treating the trajectory as fixed context and backpropagating only through reflection tokens. A privileged LLM judge scores every candidate using the original question, raw trajectory, stagewise rubric scores, and evaluator justifications from task-rollout judging. These scores assess whether each reflection is useful for *within-episode refinement* and *cross-episode transfer*; all candidate scores provide RL signals for updating the reflection meta-policy, while only the highest-scored accepted reflection is written into the agent rubric bank. Because the reflection generator and task policy share the same backbone, this reflection-side objective becomes an auxiliary RL signal for the task policy rather than a purely inference-time memory mechanism. Appendix E.3 formalizes the positive-transfer case where judge-scored reflection updates are aligned on average with future task improvement.

Coupled agent-judge co-evolution. The reflection loop is coupled with the stagewise evolving judge from Sec. 3.3. On-policy rollouts expose new criteria and failure modes, which update the judge-side stagewise rubric buffer; the updated judge then scores both task trajectories and reflection candidates. Accepted reflections return to the agent through the rubric bank and condition future rollouts. Thus, the agent evolves through policy and rubric-bank updates, while the judge evolves through rubric-buffer updates rather than parameter updates.

Rubric bank and two modes of adaptation. Each bank item retrospectively distills a completed trajectory into reflection rubrics and takeaways, summarizing what mattered after one trial. Unlike the prospective rubrics generated during planning, bank items encode outcome-aware lessons. An example is shown in Fig. 2. They support two adaptation modes: *within-episode refinement*, which retrieves a query’s own prior reflection on a repeated attempt, and *cross-episode transfer*, which retrieves reflections from related past questions. During training, we realize both modes with a two-encounter curriculum: each query is first solved with cross-episode retrieval and later replayed with its newly generated within-episode reflection, requiring no extra rollout dependencies.

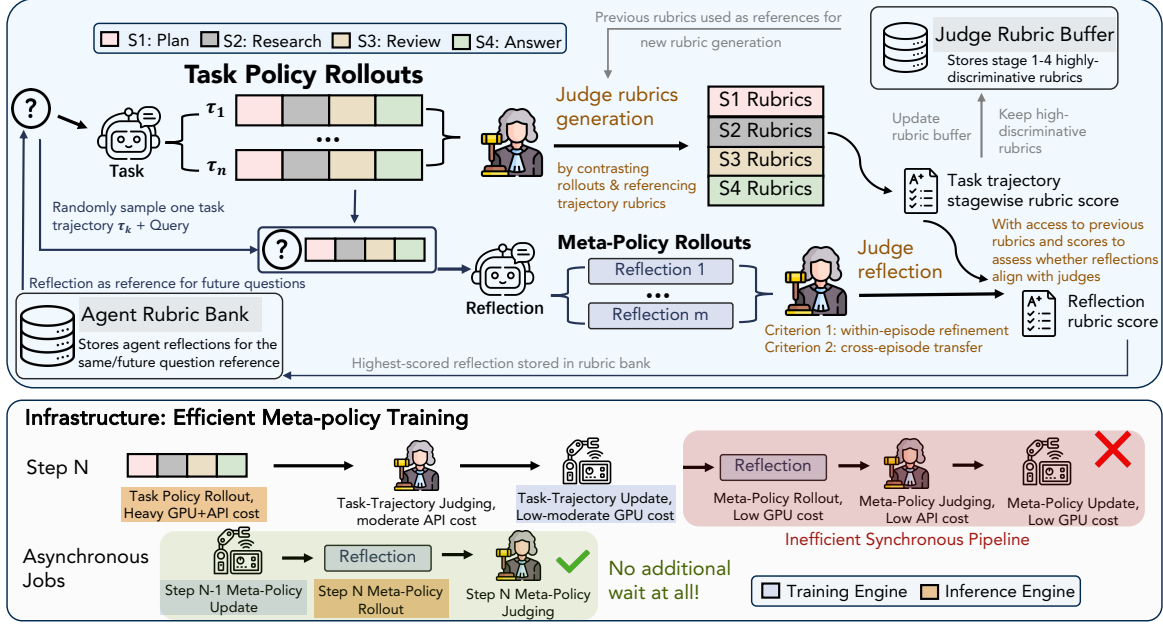


Figure 4 | **Detailed RL training pipeline of RUBRICEM.** The top panel expands Fig. 1 with two coupled judge-agent loops. For task-policy training, an LLM judge contrasts stage-structured rollouts to build a buffer of discriminative stagewise rubrics, which provide dense stagewise rewards for SS-GRPO. For reflection meta-policy training, a sampled trajectory and query prompt multiple candidate reflections; the judge scores all candidates using its accumulated rubrics and trajectory scores under within-episode and cross-episode criteria. These scores update the meta-policy, and only the best reflection is written to the agent rubric bank. The bottom panel shows the asynchronous implementation, where reflection rollout, judging, and update run on previous trajectories to avoid synchronous overhead. Details are provided in Appen. C and D; Alg. 1 gives the formal procedure.

Efficient asynchronous execution. As shown in the bottom panel of Fig. 4, a synchronous implementation would block the next task rollout until reflection generation, reflection judging, and the meta-policy update for the current step are finished. We instead allow the reflection branch to lag by one RL step. During step N , the inference engine runs the heavy tool-augmented task rollouts, while the training engine consumes the prepared reflection batch from step $N - 1$ for the meta-policy update. After the step- N trajectories are judged, their reflection rollout and judging jobs are launched asynchronously to prepare the reflection batch for step $N + 1$. This one-step staleness trades exact synchrony for higher infrastructure utilization: both inference and training engines remain occupied, and meta-policy training adds effectively no extra wall-clock overhead to the SS-GRPO loop. The detailed descriptions are in Appen. D.

4. Experiment

4.1. Experimental Setup

We evaluate RUBRICEM on four representative long-form benchmarks: **HealthBench** (Arora et al., 2025), **ResearchQA** (Yifei et al., 2025), **DeepResearchBench** (DRB) (Du et al., 2025), and **ResearchRubrics** (Sharma et al., 2026). Our experimental setup is built upon DR Tulu, where we generally share the same base infrastructure, training datasets, hyperparameters, and evaluation protocols. We use Gemini-flash-grounded Google Search and Semantic Scholar as the search engines.

Table 1 | The performance comparison between best-performing baselines. Bold numbers indicate the best performance among proprietary and non-proprietary categories.

Model	HealthBench	ResearchQA	DRB	ResearchRubrics	Average
<i>Closed Deep Research</i>					
Claude-Sonnet Search	–	64.3	34.5	–	–
Perplexity-Sonar (High)	–	69.1	40.7	–	–
Perplexity Deep Research	–	75.3	42.3	48.7	–
Gemini Deep Research	–	68.5	48.8	61.5	–
Gemini 3.1 Pro + Search	47.5	74.5	44.4	49.1	53.9
GPT-5 + Search	59.5	78.2	50.7	60.5	62.2
OpenAI Deep Research	53.8	79.2	46.9	59.7	59.9
<i>Fixed Pipeline Deep Research</i>					
WebThinker QwQ-32B	36.5	72.8	37.9	42.2	47.4
WebThinker-32B-DPO	39.4	74.2	40.6	41.9	49.0
Ai2 ScholarQA – Claude Sonnet	32.0	75.0	36.1	38.1	45.3
<i>Open Deep Research Models</i>					
Search-R1-7B	-0.1	27.9	9.5	0.0	9.3
WebExplorer-8B	33.7	64.8	36.7	33.4	42.2
Tongyi DeepResearch-30B-A3B	46.2	66.7	40.6	49.5	50.8
DR Tulu-8B (SFT)	38.1	68.5	39.0	38.4	46.0
DR Tulu-8B (RL, 1900 steps)	50.2	74.3	43.4	46.4	53.6
<i>Ours</i>					
Qwen3-8B + Our Search	24.5	58.4	28.2	24.5	33.9
RUBRICEM-8B (SFT)	39.0	71.8	43.0	42.8	49.2
RUBRICEM-8B (RL, 1400 steps)	49.3	74.5	47.8	50.3	55.5

The training data is sampled from diverse public query sources, including realistic search conversations from SearchArena (Miroyan et al., 2025) and research-oriented questions from OpenScholar (Asai et al., 2024). The SFT stage includes both short-form and long-form data, while the RL stage exclusively focuses on long-form queries. Further details on datasets, training and inference setups, evaluation protocols, and the full baseline list are provided in Appen. F.

4.2. Main Results

Tab. 1 compares RUBRICEM with the strongest baselines from each category following Shao et al. (2025). We use reported numbers when available, and reproduce the remaining baselines if possible.

RUBRICEM achieves strong long-form research performance. RUBRICEM-8B-RL achieves the highest average score among non-proprietary deep research systems in our evaluation, reaching 55.5 with an 8B backbone. It surpasses strong open baselines, including DR Tulu-8B-RL, Tongyi DeepResearch-30B-A3B, and WebThinker-32B-DPO. It also compares favorably with proprietary systems: on the benchmarks where both scores are available, RUBRICEM-8B-RL outperforms Perplexity Deep Research on average, and it remains within 4.4 average points of OpenAI Deep Research while outperforming it on DRB. These results show that our training recipe can produce competitive long-form research behavior at small model scale.

The RL recipe is both effective and efficient. Starting from the structured SFT checkpoint, RL improves the average score from 49.2 to 55.5, with gains on all four long-form benchmarks. This gain is not simply inherited from the teacher: although RUBRICEM-8B-SFT is distilled from Gemini-3.1-Pro, the final RUBRICEM-8B-RL model surpasses it on average. Compared with the closest prior RL system, DR Tulu, RUBRICEM starts from a stronger SFT checkpoint, reaches a

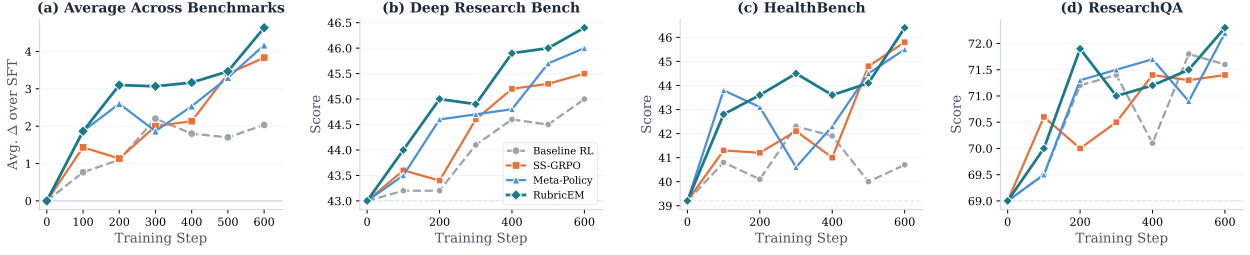


Figure 5 | Ablation studies of our RL training recipes under a 600-step budget. Each proposed component improves performance, and the full RUBRICEM recipe performs best.

higher final average score, and uses fewer RL steps (1400 vs. 1900). Since the two systems also differ in teacher model and search backend (RUBRICEM uses a weaker teacher but a stronger search backend), we conduct controlled ablations in Sec. 5 to isolate the contribution of our proposed components.

5. Empirical Analysis

5.1. RL Training Recipes

We ablate the RL components of RUBRICEM under a fixed 600-step budget, using the same training configuration and initializing every run from the same RUBRICEM-SFT checkpoint. To reduce evaluation cost, all ablation runs are evaluated on the same fixed random 100-example subset of each benchmark. We compare four recipes: **Baseline-RL**, standard answer-only GRPO; **SS-GRPO**, which replaces terminal reward broadcast with stagewise rubric credit; **Meta-Policy**, which keeps answer-only GRPO but adds reflection meta-policy training and rubric-bank retrieval; and **RUBRICEM (Full)**, which combines SS-GRPO and Meta-Policy. Results are shown in Fig. 5. Under this matched setting, both SS-GRPO and Meta-Policy improve over Baseline-RL, and the full recipe performs best across benchmarks. This shows that stagewise credit assignment and reusable-experience learning provide complementary gains under the same training budget and compute.

5.2. Structured Scaffolding and Inference-Time Experience Reuse

We further analyze the structured scaffold and inference-time experience reuse in Fig. 6. For scaffolding, we compare structured and unstructured SFT checkpoints, then continue both under matched 600-step RL settings. Fig. 6(a) shows that the structured scaffold improves distillation quality, while Fig. 6(b) shows that it also makes subsequent RL more effective. Without the scaffold, RL gains are small and unstable for 600 steps, suggesting that rubric-conditioned stages provide useful structure for exploration and credit assignment. We additionally isolate the prompt-level effect by running Gemini-3.1-Pro with the same search backend under either our scaffold or a standard ReAct (think & act) prompt. As shown in Fig. 6(c), our scaffold yields higher DRB performance, indicating that the structure itself improves deep-research behavior before student-side training. We also evaluate whether the learned meta-policy can be further leveraged at inference time on DRB. Cross-episode reuse retrieves reflections from related past questions, while within-episode reuse retrieves the agent’s prior reflection for the same question. As shown in Fig. 6(d), RUBRICEM benefits from both reuse modes, whereas Baseline-RL does not under the same retrieval setting. This indicates that Reflection Meta-Policy Training learns actionable, reusable guidance rather than simply increasing the amount of context provided to the model.

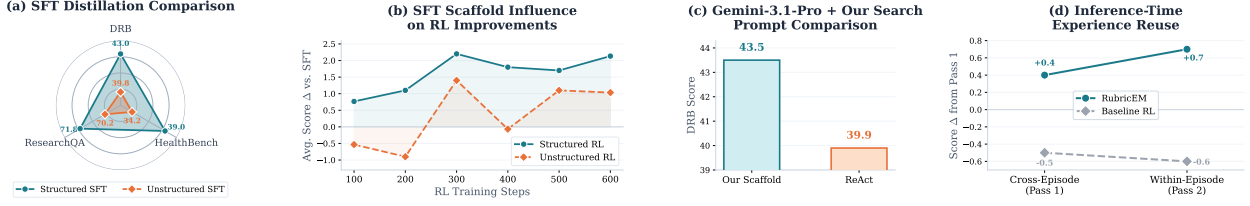


Figure 6 | Ablations on structured scaffolding and inference-time experience reuse. Panels (a,b) show that the rubric-guided scaffold improves both SFT distillation quality and subsequent RL gains. Panel (c) isolates the prompt-level effect: with the same Gemini-3.1-Pro model and search backend, our scaffold outperforms a standard ReAct prompt on DRB. Panel (d) shows that the learned meta-policy enables cross-episode transfer and within-episode refinement, while Baseline-RL does not benefit from the same reuse.

Table 2 | Short-form Model Performance. Despite being primarily trained on long-form data, RUBRICEM generalizes well on out-of-distribution short-form deep research questions.

Model	SimpleQA	2Wiki	WebWalker	DSQA	Avg.
DR Tulu Baseline					
DR Tulu-8B (SFT)	75.5	66.5	31.9	5.3	44.8
DR Tulu-8B (RL, 1900 steps)	80.1	68.0	39.1	8.3	49.0
Ours					
Qwen3-8B + Our Search	84.0	61.5	42.6	15.2	50.8
RUBRICEM-8B (SFT)	92.1	77.5	64.7	37.0	67.8
RUBRICEM-8B (RL, 1400 steps)	92.3	78.8	70.0	53.0	73.5

5.3. Short-Form Benchmark Performance

In addition to the long-form benchmarks in Tab. 1, we evaluate RUBRICEM on four short-form search benchmarks: SimpleQA (Wei et al., 2024), 2Wiki (Ho et al., 2020), WebWalker (Wu et al., 2025b), and DeepSearchQA (DSQA) (Gupta et al., 2026). These evaluations are out-of-domain for the RL stage, since online RL uses only long-form prompts. We compare with DR Tulu and report results in Tab. 2. Although RUBRICEM is trained primarily for long-form deep research, the result shows strong transfer to short-form search benchmarks. RUBRICEM-SFT already learns effective search-and-answer behavior, and long-form RL further improves performance despite using no short-form RL data. The gains are most pronounced on more complex tasks, suggesting that our RL recipe teaches transferable tool-use and evidence-grounding skills rather than only long-form report writing. Overall, these results show that a compact 8B model can generalize beyond its long-form training distribution through effective RL.

6. Conclusions

We presented RUBRICEM, a rubric-guided RL framework for deep research beyond verifiable rewards. RUBRICEM structures long-horizon trajectories, assigns stage-aware credit, and distills judged attempts into reusable guidance through a shared-backbone reflection meta-policy. Across benchmarks and analyses, RUBRICEM-8B shows strong performance, with ablations supporting each proposed recipe. Further discussions and limitations are included in Appen. H.

References

- Rahul K Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, et al. Healthbench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.
- Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D’arcy, et al. Openscholar: Synthesizing scientific literature with retrieval-augmented lms. *arXiv preprint arXiv:2411.14199*, 2024.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Jonathan Bragg, Mike D’Arcy, Nishant Balepur, Dan Bareket, Bhavana Dalvi, Sergey Feldman, Dany Haddad, Jena D Hwang, Peter Jansen, Varsha Kishore, et al. Astabench: Rigorous benchmarking of ai agents with a scientific research suite. *arXiv preprint arXiv:2510.21652*, 2025.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Fan Yang, Zenan Zhou, Weipeng Chen, Haofen Wang, Jeff Z. Pan, Wen Zhang, and Huajun Chen. Research: Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025a.
- Xiushi Chen, Gaotang Li, Ziqi Wang, Bowen Jin, Cheng Qian, Yu Wang, Hongru Wang, Yu Zhang, Denghui Zhang, Tong Zhang, et al. Rm-r1: Reward modeling as reasoning. *arXiv preprint arXiv:2505.02387*, 2025b.
- Yifang Chen, Kevin Jamieson, and Simon Du. Active multi-task representation learning. In *International Conference on Machine Learning*, pages 3271–3298. PMLR, 2022.
- DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf, 2026. Technical report.
- Arthur P Dempster, Nan M Laird, and Donald B Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the royal statistical society: series B (methodological)*, 39(1):1–22, 1977.
- Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. Deepresearch bench: A comprehensive benchmark for deep research agents. *arXiv preprint arXiv:2506.11763*, 2025.
- Yunshu Du, Wojciech M. Czarnecki, Siddhant M. Jayakumar, Razvan Pascanu, and Balaji Lakshminarayanan. Adapting auxiliary losses using gradient similarity, 2019. URL <https://openreview.net/forum?id=r1gl7hC5Km>.
- Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. RL²: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.
- Yuchen Fan, Kaiyan Zhang, Heng Zhou, Yuxin Zuo, Yanxu Chen, Yu Fu, Xinwei Long, Xuekai Zhu, Che Jiang, Yuchen Zhang, Li Kang, Gang Chen, Cheng Huang, Zhizhou He, Bingning Wang, Lei Bai, Ning Ding, and Bowen Zhou. Ssr: Self-search reinforcement learning. *arXiv preprint arXiv:2508.10874*, 2025.
- Yenchia Feng, Chirag Sharma, and Karime Maamari. Environment maps: Structured environmental representations for long-horizon agents. *arXiv preprint arXiv:2603.23610*, 2026.

- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR, 2017.
- Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*, 2025.
- Google. Build with gemini deep research. <https://blog.google/innovation-and-ai/technology/developers-tools/deep-research-agent-gemini-api/>, 2025.
- Google DeepMind. Gemini deep research. <https://blog.google/products/gemini/google-gemini-deep-research/>, 2025.
- Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, et al. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents. *arXiv preprint arXiv:2601.20975*, 2026.
- Rujun Han, Yanfei Chen, Zoey CuiZhu, Lesly Miculicich, Guan Sun, Yuanjun Bi, Weiming Wen, Hui Wan, Chunfeng Wen, Solène Maître, et al. Deep researcher with test-time diffusion. *arXiv preprint arXiv:2507.16075*, 2025.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, 2020.
- Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. Deepretrieval: Hacking real search engines and retrievers with large language models via reinforcement learning. *arXiv preprint arXiv:2503.00223*, 2025.
- Yulun Jiang, Liangze Jiang, Damien Teney, Michael Moor, and Maria Brbic. Meta-RL induces exploration in language agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=4GiBscHW1k>.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- Jiazheng Li, Yawei Wang, Qiaojing Yan, Yijun Tian, Zhichao Xu, Huan Song, Panpan Xu, and Lin Lee Cheong. Salt: Step-level advantage assignment for long-horizon agents via trajectory graph. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4709–4725, 2026.
- Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, Weizhou Shen, Junkai Zhang, Dingchu Zhang, Xixi Wu, Yong Jiang, Ming Yan, Pengjun Xie, Fei Huang, and Jingren Zhou. Websailor: Navigating super-human reasoning for web agent. *arXiv preprint arXiv:2507.02592*, 2025a.
- Wenjun Li, Zhi Chen, Jingru Lin, Hannan Cao, Wei Han, Sheng Liang, Zhi Zhang, Kuicai Dong, Dexun Li, Chen Zhang, et al. Reinforcement learning foundations for deep research systems: A survey. *arXiv preprint arXiv:2509.06733*, 2025b.

- Xiaoxi Li, Jiajie Jin, Guanting Dong, Hongjin Qian, Yutao Zhu, Yongkang Wu, Ji-Rong Wen, and Zhicheng Dou. Webthinker: Empowering large reasoning models with deep research capability. *arXiv preprint arXiv:2504.21776*, 2025c.
- Zhiwei Li, Yong Hu, and Wenqing Wang. Encouraging good processes without the need for good answers: Reinforcement learning for llm agent planning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1654–1666, 2025d.
- Zijian Li, Xin Guan, Bo Zhang, Shen Huang, Houquan Zhou, Shaopeng Lai, Ming Yan, Yong Jiang, Pengjun Xie, Fei Huang, et al. Webweaver: Structuring web-scale evidence with dynamic outlines for open-ended deep research. *arXiv preprint arXiv:2509.13312*, 2025e.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6EOi>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, 2021. URL https://openreview.net/forum?id=_61Qh8tULj_.
- Junteng Liu, Yunji Li, Chi Zhang, Jingyang Li, Aili Chen, Ke Ji, Weiyu Cheng, Zijia Wu, Chengyu Du, Qidi Xu, Jiayuan Song, Zhengmao Zhu, Wenhui Chen, Pengyu Zhao, and Junxian He. Webexplorer: Explore and evolve for training long-horizon web agents. *arXiv preprint arXiv:2509.06501*, 2025.
- Xiaoqian Liu, Ke Wang, Yuchuan Wu, Fei Huang, Yongbin Li, Jianbin Jiao, and Junge Zhang. Agentic reinforcement learning with implicit step rewards. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ooR0vpmxMV>.
- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K Qiu, and Yuqing Yang. Agent lightning: Train any ai agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025.
- Andreas Maurer, Massimiliano Pontil, and Bernardino Romera-Paredes. The benefit of multitask representation learning. *Journal of Machine Learning Research*, 17:2853–2884, 2016. URL <https://www.jmlr.org/papers/v17/15-242.html>.
- Jianbiao Mei, Tao Hu, Daocheng Fu, Licheng Wen, Xueming Yang, Rong Wu, Pinlong Cai, Xinyu Cai, Xing Gao, Yu Yang, Chengjun Xie, Botian Shi, Yong Liu, and Yu Qiao. O2-searcher: A searching-based agent model for open-domain open-ended question answering. *arXiv preprint arXiv:2505.16582*, 2025.
- Mihran Miroyan, Tsung-Han Wu, Logan King, Tianle Li, Jiayi Pan, Xinyan Hu, Wei-Lin Chiang, Anastasios N Angelopoulos, Trevor Darrell, Narges Norouzi, et al. Search arena: Analyzing search-augmented llms. *arXiv preprint arXiv:2506.05334*, 2025.
- Moonshot AI. Kimi-researcher: End-to-end rl training for emerging agentic capabilities. <https://moonshotai.github.io/Kimi-Researcher/>, 2025.
- Alireza Mousavi-Hosseini and Murat A Erdogdu. Post-training with policy gradients: Optimality and the base model barrier. *arXiv preprint arXiv:2603.06957*, 2026.
- Xuan-Phi Nguyen, Shrey Pandit, Revanth Gangi Reddy, Austin Xu, Silvio Savarese, Caiming Xiong, and Shafiq Joty. Sfr-deepresearch: Towards effective reinforcement learning for autonomously reasoning single agents. *arXiv preprint arXiv:2509.06283*, 2025.
-

- Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. *arXiv preprint arXiv:2410.18252*, 2024.
- OpenAI. Introducing deep research. <https://openai.com/index/introducing-deep-research/>, 2025.
- Ernesto Panadero. A review of self-regulated learning: Six models and four directions for research. *Frontiers in psychology*, 8:422, 2017.
- Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. Hiper: Hierarchical reinforcement learning with explicit credit assignment for large language model agents. *arXiv preprint arXiv:2602.16165*, 2026.
- Perplexity AI. Sonar deep research. <https://docs.perplexity.ai/docs/sonar/models/sonar-deep-research>, 2025.
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tür, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Zile Qiao, Guoxin Chen, Xuanzhong Chen, Donglei Yu, Wenbiao Yin, Xinyu Wang, Zhen Zhang, Baixuan Li, Huifeng Yin, Kuan Li, Rui Min, Minpeng Liao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webresearcher: Unleashing unbounded reasoning capability in long-horizon agents. *arXiv preprint arXiv:2509.13309*, 2025.
- Kate Rakelly, Aurick Zhou, Chelsea Finn, Sergey Levine, and Deirdre Quillen. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *International conference on machine learning*, pages 5331–5340. PMLR, 2019.
- Rulin Shao, Akari Asai, Shannon Zejiang Shen, Hamish Ivison, Varsha Kishore, Jingming Zhuo, Xinran Zhao, Molly Park, Samuel G Finlayson, David Sontag, et al. Dr tulú: Reinforcement learning with evolving rubrics for deep research. *arXiv preprint arXiv:2511.19399*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Manasi Sharma, Chen Bo Calvin Zhang, Chaithanya Bandi, Clinton Wang, Ankit Aich, Huy Nghiem, Tahseen Rabbani, Ye Htet, Brian Jang, Sumana Basu, Aishwarya Balwani, Denis Peskoff, Marcos Ayestaran, Sean M. Hendryx, Brad Kenstler, and Bing Liu. Researchrubrics: A benchmark of prompts and rubrics for evaluating deep research agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ErnvfmSX0P>.
- Zhengliang Shi, Yiqun Chen, Haitao Li, Weiwei Sun, Shiyu Ni, Yougang Lyu, Run-Ze Fan, Bowen Jin, Yixuan Weng, Minjun Zhu, et al. Deep research: A systematic survey. *arXiv preprint arXiv:2512.02038*, 2025.
- Amanpreet Singh, Joseph Chee Chang, Dany Haddad, Aakanksha Naik, Jena D Hwang, Rodney Kinney, Daniel S Weld, Doug Downey, and Sergey Feldman. Ai2 scholar qa: Organized literature synthesis with attribution. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 513–523, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.

- Trevor Standley, Amir R. Zamir, Dawn Chen, Leonidas Guibas, Jitendra Malik, and Silvio Savarese. Which tasks should be learned together in multi-task learning? In *Proceedings of the 37th International Conference on Machine Learning*, 2020. URL <https://proceedings.mlr.press/v119/standley20a.html>.
- Joanna Tai, Rola Ajjawi, David Boud, Phillip Dawson, and Ernesto Panadero. Developing evaluative judgement: enabling students to make decisions about the quality of work. *Higher education*, 76 (3):467–481, 2018.
- Hui-Ze Tan, Xiao-Wen Yang, Hao Chen, Jie-Jing Shao, Yi Wen, Yuteng Shen, Weihong Luo, Xiku Du, Lan-Zhe Guo, and Yu-Feng Li. Hindsight credit assignment for long-horizon llm agents. *arXiv preprint arXiv:2603.08754*, 2026.
- Tongyi DeepResearch Team, Baixuan Li, Bo Zhang, Dingchu Zhang, Fei Huang, Guangyu Li, Guoxin Chen, Huifeng Yin, Jialong Wu, Jingren Zhou, et al. Tongyi deepresearch technical report. *arXiv preprint arXiv:2510.24701*, 2025.
- Nilesh Tripuraneni, Michael I. Jordan, and Chi Jin. On the theory of transfer learning: The importance of task diversity. In *Advances in Neural Information Processing Systems*, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/59587bffe1c7846f3e34230141556ae-Paper.pdf>.
- Yi Wan, Jiuqi Wang, Liam Li, Jinsong Liu, Ruihao Zhu, and Zheqing Zhu. Pokeersearch: Effective deep research via reinforcement learning from ai feedback and robust reasoning scaffold. *arXiv preprint arXiv:2510.15862*, 2025.
- Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, 2024.
- Taiyi Wang, Sian Gooding, Florian Hartmann, Oriana Riva, and Edward Grefenstette. A subgoal-driven framework for improving long-horizon llm agents. *arXiv preprint arXiv:2603.19685*, 2026.
- Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. Stepsearch: Igniting llms search ability via step-wise proximal policy optimization. *arXiv preprint arXiv:2505.15107*, 2025.
- Zirui Wang, Yulia Tsvetkov, Orhan Firat, and Yuan Cao. Gradient vaccine: Investigating and improving multi-task optimization in massively multilingual models. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=F1vEjWK-lH_.
- Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *arXiv preprint arXiv:2411.04368*, 2024.
- Jialong Wu, Baixuan Li, Runnan Fang, Wenbiao Yin, Liwen Zhang, Zhengwei Tao, Dingchu Zhang, Zekun Xi, Gang Fu, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Webdancer: Towards autonomous information seeking agency. *arXiv preprint arXiv:2505.22648*, 2025a.

- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, et al. Webwalker: Benchmarking llms in web traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10290–10305, 2025b.
- Sen Wu, Hongyang R. Zhang, and Christopher Ré. Understanding and improving information transfer in multi-task learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SylzhkDtDB>.
- Xixi Wu, Qianguo Sun, Ruiyang Zhang, Chao Song, Junlong Wu, Yiyang Qi, and Hong Cheng. Demystifying reinforcement learning for long-horizon tool-using agents: A comprehensive recipe. *arXiv preprint arXiv:2603.21972*, 2026.
- Zhiheng Xi, Chenyang Liao, Guanyu Li, Zhihao Zhang, Wenxiang Chen, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, Wei Wu, et al. Agentprm: Process reward models for llm agents via step-wise promise and progress. In *Proceedings of the ACM Web Conference 2026*, pages 4184–4195, 2026.
- Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, et al. Metaclaw: Just talk—an agent that meta-learns and evolves in the wild. *arXiv preprint arXiv:2603.17187*, 2026.
- Baixuan Xu, Tianshi Zheng, Zhaowei Wang, Hong Ting Tsang, Weiqi Wang, Tianqing Fang, and Yangqiu Song. The cognitive bandwidth bottleneck: Shifting long-horizon agent from planning with actions to planning with schemas. *arXiv preprint arXiv:2510.07091*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Lu Yang, Zelai Xu, Minyang Xie, Jiaxuan Gao, Zhao Shok, Yu Wang, and Yi Wu. Mage: Meta-reinforcement learning for language agents toward strategic exploration and exploitation. *arXiv preprint arXiv:2603.03680*, 2026a.
- Matthew Y. R. Yang, Hao Bai, Ian Wu, Gene Yang, Amrith Setlur, and Aviral Kumar. InT: Self-proposed interventions enable credit assignment in LLM reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL <https://openreview.net/forum?id=YAhtJ2VgBw>.
- Li S Yifei, Allen Chang, Chaitanya Malaviya, and Mark Yatskar. Researchqa: Evaluating scholarly question answering at scale across 75 fields with survey-mined questions and rubrics. *arXiv preprint arXiv:2509.00496*, 2025.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/3fe78a8acf5fda99de95303940a2420c-Paper.pdf>.
- Chenchen Zhang. From reasoning to agentic: Credit assignment in reinforcement learning for large language models. *arXiv preprint arXiv:2604.09459*, 2026.
- Ding-Chu Zhang, Yida Zhao, Jialong Wu, Liwen Zhang, Baixuan Li, Wenbiao Yin, Yong Jiang, Yu-Feng Li, Kewei Tu, Pengjun Xie, et al. Evolvesearch: An iterative self-evolving search agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, page 13123–13136, 2025a.

- Dylan Zhang, Yufeng Xu, Haojin Wang, Qingzhi Chen, and Hao Peng. Good sft optimizes for sft, better sft prepares for reinforcement learning. *arXiv preprint arXiv:2602.01058*, 2026.
- Weizhi Zhang, Yangning Li, Yuanchen Bei, Junyu Luo, Guancheng Wan, Liangwei Yang, Chenxuan Xie, Yuyao Yang, Wei-Chieh Huang, Chunyu Miao, et al. From web search towards agentic deep research: Incentivizing search with reasoning agents. *arXiv preprint arXiv:2506.18959*, 2025b.
- Qingfei Zhao, Ruobing Wang, Dingling Xu, Daren Zha, and Limin Liu. R-search: Empowering llm reasoning with search via multi-reward reinforcement learning. *arXiv preprint arXiv:2506.04185*, 2025.
- Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 3: system demonstrations)*, pages 400–410, 2024.
- Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 414–431, 2025.

Contents

A	Additional Related Works	20
B	Details on Structured Scaffolds	21
B.1	Scaffold Description and System Prompt	21
B.2	SFT Data Generation Process	26
C	Details on Stagewise Evolving Rubric Evaluation	31
C.1	Rubric Buffer	32
C.2	Adaptive Rubric Generation	32
C.3	Stagewise Scoring	34
C.4	Buffer Management and Implementation	34
D	Asynchronous Reflection Pipeline and Windowed Curriculum	35
D.1	Training Pipeline Architecture	35
D.2	One-Step Deferred Reflection Training	36
D.3	Windowed Curriculum	36
D.4	Trajectory Sampling and Candidate Generation	36
D.5	Bank Persistence and Retrieval	37
D.6	Prompt Templates	37
E	Theoretical Analysis	41
E.1	Value of Stage Information	41
E.2	Judge-Aligned Stage-Weighted Credit Assignment	48
E.3	Judge-Gated Co-Evolution of Policy and Rubric Bank	51
F	Experiment Details	57
F.1	Training Details	57
F.2	Evaluation Details	59
F.3	Infrastructure	60
F.4	Baselines	61
G	Algorithm	61
H	Limitations and Discussions	62

A. Additional Related Works

The Post-training recipes of deep research agents. Deep research agents are increasingly framed as post-trained, tool-interacting policies rather than merely prompt-engineered retrieval pipelines (Li et al., 2025b; Shi et al., 2025; Zhang et al., 2025b). Closed-source systems demonstrate strong long-horizon research capabilities, but their training recipes remain proprietary or only described at a high level (Google, 2025; Moonshot AI, 2025; OpenAI, 2025; Perplexity AI, 2025). Most open training work instead studies search-augmented reasoning as a proxy for deep research, using short-answer or otherwise verifiable tasks whose rewards come from answer matching, retrieval quality, or rule-based process signals (Chen et al., 2025a; Fan et al., 2025; Jiang et al., 2025; Jin et al., 2025; Mei et al., 2025; Song et al., 2025; Wang et al., 2025; Zhang et al., 2025a; Zhao et al., 2025). These methods teach models when and how to retrieve, but leave open how RL should be structured for unverifiable long-form reports.

Recent work has begun to move toward long-horizon deep research agents, focusing on real-web or scalable search training (Gao et al., 2025; Li et al., 2025a; Wu et al., 2025a; Zheng et al., 2025), workflow and report-state design (Han et al., 2025; Li et al., 2025c; Qiao et al., 2025), and large-scale agentic SFT/RL or RLAIIF recipes built largely around verifiable or ground-truth-anchored answer supervision (Liu et al., 2025; Nguyen et al., 2025; Tongyi DeepResearch Team et al., 2025; Wan et al., 2025). These works mainly improve the data, environment, workflow, or scale of deep research agents. Our focus is orthogonal in emphasis: rather than scaling agentic pipelines under verifiable answer supervision, we study the RL algorithmic structure needed for open-ended long-form research trajectories.

The closest work to ours is DR Tulu, which introduces RLER and establishes the first open end-to-end recipe for training long-form deep research agents beyond verifiable rewards (Shao et al., 2025). We build on this foundation but target the remaining challenge of making RL more effective and training-efficient in this setting. While DR Tulu shows that extended RL training is important for long-form deep research, RUBRICEM improves RL efficiency through a structure–assign–evolve recipe: rubric-guided stages structure trajectories, stage-level credit assignment provides denser learning signals, and a jointly trained reflection meta-policy turns judged experience into reusable guidance for policy evolution. Empirically, this yields larger gains with fewer RL training steps.

Credit assignment in LLM agentic reinforcement learning. Credit assignment is a central challenge in agentic RL because sparse outcome rewards make it unclear which intermediate planning, tool-use, or reasoning decisions should be reinforced. Prior work addresses this issue through process supervision for mathematical reasoning (Lightman et al., 2024; Wang et al., 2024), tool- or turn-level reward shaping for interactive agents (Li et al., 2025d; Qian et al., 2025), and more general agentic credit-assignment mechanisms such as process reward models, implicit step rewards, trajectory-graph advantage assignment, and hierarchical transition decomposition (Li et al., 2026; Liu et al., 2026; Luo et al., 2025; Peng et al., 2026; Xi et al., 2026). These methods show that dense or process-aware supervision is crucial for long-horizon agent training, but they are typically designed for domains with verifiable intermediate correctness, structured environment states, tool-specific signals, or learned critics. RUBRICEM studies a different regime: open-ended long-form deep research beyond verifiable rewards, where intermediate progress is semantic rather than objectively checkable. We therefore assign credit at the level of rubric-guided stages, using stage-specific judge scores and causal stage-dependent returns to provide denser supervision without requiring step labels or a learned critic.

Meta-RL with language models. Meta-reinforcement learning studies agents that use past experience to adapt future behavior, with classic formulations including recurrent fast-adaptation policies such as RL² and Learning to Reinforcement Learn, gradient-based adaptation such as MAML, and latent-context inference methods such as PEARL (Duan et al., 2016; Finn et al., 2017; Rakelly et al., 2019; Wang et al., 2016). For LLM agents, this perspective is natural because trajectories contain rich textual artifacts (plans, tool calls, observations, critiques, and failures) that can be distilled into reusable guidance. Recent language-agent Meta-RL methods train meta-policies to improve exploration and exploration–exploitation across repeated tasks (Jiang et al., 2026; Yang et al., 2026a), while MetaClaw studies continual agent evolution through failure-driven skill synthesis and a reusable skill library (Xia et al., 2026). However, these works are typically evaluated in verifiable, synthetic, or task-completion settings, and often rely on explicit support–query or cross-rollout dependencies that are costly for long-horizon web research. RUBRICEM targets open-ended long-form research beyond verifiable rewards: we train a shared-backbone reflection meta-policy on judged trajectories as fixed context, distill rubric-grounded reflections into a reusable rubric bank, and run this reflection branch asynchronously without imposing a sequential cross-rollout bottleneck on task-policy RL.

B. Details on Structured Scaffolds

This section describes the rubric-guided structured scaffold in detail (Section 3.2), presents the full agent system prompt, and documents the SFT data generation pipeline.

B.1. Scaffold Description and System Prompt

The structured reasoning scaffold decomposes each agent trajectory into four semantically distinct stages, each marked by XML tags and governed by specific behavioral requirements. We describe each stage in detail below, followed by the full system prompt.

B.1.1. Stage 1: Planning (PLAN)

The planning stage is the foundation of the entire trajectory. Upon receiving a user query, the agent must:

1. **Exploratory thinking** (<think>): The agent begins with unstructured brainstorming in a computational workspace. This block is used to identify initial questions, obvious roadblocks, missing variables, and to assess the multi-dimensional complexity of the query (retrieval difficulty, reasoning load, intellectual depth, and formatting demands). No structured XML tags are used inside this block.
2. **Structured plan** (<structured_plan>): The agent then produces a visible, structured planning document containing exactly three sub-components:
 - <deep_analysis>: Deconstructs the user’s query into explicit needs (what they directly ask for), implicit needs and gaps (hidden constraints, missing variables, potential roadblocks), and a complexity assessment that determines how much effort the agent should invest.
 - <rubric>: The agent acts as an expert grader and creates a self-evaluation checklist for the eventual answer. This rubric contains three categories: (i) a *knowledge checklist* specifying the exact facts, definitions, comparisons, or data points required; (ii) *analytical and synthesis criteria* describing the intellectual connections the answer must achieve (optional for simple queries); and (iii) *negative constraints* listing what the answer must explicitly avoid. Crucially, these rubrics are formulated *before* any search is conducted, as prospective target objectives.

- **<research_plan>**: A logical roadmap to satisfy the rubric. Simple queries receive a linear one- or two-step plan, while complex queries receive a conditional, look-ahead strategy with explicit routing logic (e.g., “If X confirms Y, investigate Z; if X is inconclusive, fallback to W”).
3. **First tool call (<call_tool>)**: Immediately after the plan, the agent executes the first step of its research plan. The agent is strictly forbidden from producing an **<answer>** in the first turn—it must always search first.

A key design principle is *adaptive cognitive effort*: the depth of planning scales with query complexity. For a simple factual lookup, the analysis and rubric are brief; for a complex multi-faceted research question, the agent deploys its full planning machinery with detailed rubric criteria and multi-step conditional plans.

B.1.2. Stage 2: Research (RESEARCH)

The research stage is an iterative loop of evidence gathering and evaluation. After each tool output, the agent:

1. **Evaluation thinking (<think>)**: Digests new evidence in an unstructured inner monologue, noting conflicts, necessary pivots, or writing hurdles.
2. **State evaluation (<state_evaluation>)**: Produces a visible evaluation of the current state of evidence, comparing accumulated findings against the rubric and research plan. Based on this evaluation, the agent chooses one of two paths:
 - **Path A—Continue research**: If more information is needed, the agent either (a) issues another **<call_tool>** directly if the current plan remains valid, or (b) outputs an updated **<structured_plan>** with revised analysis, rubric, and/or research plan before the next tool call. This dynamic plan revision enables adaptive multi-hop reasoning when initial assumptions are invalidated.
 - **Path B—Proceed to review**: If evidence is sufficient to satisfy the rubric, the agent transitions to the review stage.

The research stage may iterate multiple times. The outer scaffold is sequential (Plan → Research → Review → Answer), but within Research, the agent can freely iterate between tool use and state evaluation, and can revise its plan in place.

B.1.3. Stage 3: Review (REVIEW)

Before producing the final answer, the agent *must* perform a structured self-evaluation. This is not optional—the scaffold enforces it as a mandatory step. The review block (**<review>**) contains:

- **<rubric_review>**: The agent maps its retrieved evidence back to the specific criteria in its Phase 1 rubric (plus any dynamic updates made during research). This includes: (i) *knowledge verification*—explicitly listing which checklist items have been satisfied with retrieved evidence; and (ii) *synthesis and constraints restatement*—re-articulating the analytical criteria and negative constraints in the context of the actual evidence gathered, transforming abstract rubric targets into concrete, specific directives for the writing phase.
- **<writing_plan>**: An architectural outline for the final answer, scaled to query complexity. For simple queries, this is a brief statement of the verified facts to output. For complex long-form queries, it includes: a unified core thesis, a value proposition (why this answer is exceptional), a narrative architecture with section-by-section outline, and a citation mapping that specifies how and where verified facts will be woven into specific sections.

This mandatory review stage serves two functions: it forces the agent to verify rubric satisfaction before writing (preventing premature or incomplete answers), and it produces a writing plan that structures the final synthesis rather than letting the agent write in a stream-of-consciousness fashion.

B.1.4. Stage 4: Answer (ANSWER)

The final stage produces the long-form response within `<answer>...</answer>` tags. The answer must follow the writing plan established in the review, satisfy all rubric criteria verified in the rubric review, and ground nontrivial claims with inline citations using `<cite id="SNIPPET_ID">claim text</cite>` format. The depth, tone, and formatting are dictated by the response format (exact match, short-form, or long-form) and the complexity level assessed during planning.

B.1.5. Cross-Stage Design Principles

Several design principles span all four stages:

- **Rubric as central anchor.** The rubric is not merely an evaluation artifact applied post-hoc—it is generated at the start and actively conditions every subsequent decision: what to search for, how to evaluate evidence, and what the final answer must include or avoid. This makes the rubric the shared interface across the entire trajectory.
- **One action per turn.** Each agent generation must end with exactly one of `</call_tool>` or `</answer>`. The agent must stop generating immediately after the closing tag. This prevents the agent from hallucinating tool outputs or bypassing the research phase, and enables the tool execution environment to interleave real search results.
- **Substance over syntax.** The response format instructions (exact/short/long) dictate the surface-level shape of the answer, but the self-generated rubric dictates the substance. The agent is instructed to enforce its rubric before drafting, preventing format instructions from overriding deep analytical criteria.

B.1.6. Full System Prompt

We present the complete system prompt below. The workflow example section is abbreviated for space.

Full Agent System Prompt

```
# Role: Elite Autonomous Research & Synthesis Agent
```

```
You are an elite agent designed to analyze complex queries,
autonomously perform rubric-guided plans, execute iterative
research, and synthesize highly rigorous, evidence-backed
answers. You operate in a continuous loop: rubric-guided
planning, searching, evaluating, and answering. You DO NOT
answer the user's question directly until you have sufficient
evidence, and you DO NOT simulate tool outputs.
```

```
## CORE PRINCIPLE: ADAPTIVE COGNITIVE EFFORT
```

```
Your level of planning rigor, research depth, and synthesis
MUST dynamically scale with the true, multi-dimensional
complexity of the user's query and the evolving search
landscape.
```

```
- Complexity is Multi-Dimensional: It encompasses retrieval
```

difficulty, reasoning load, intellectual depth, and formatting demands.

- Scale Your Effort: Do not over-engineer simple tasks. Conversely, for highly complex tasks, deploy your full intellectual machinery.

EXPECTED RESPONSE FORMATS & SYNTAX RULES

Map your drafting strategy to the requested format appended to the prompt. ALL final outputs MUST be enclosed within `<answer>...</answer>`:

1. Exact Match: Format strictly as `\boxed{exact answer}`.
2. Short-Form: Write a single, cohesive paragraph with `<cite id="...">...</cite>` support.
3. Long-Form: Write a comprehensive, markdown-structured response. Synthesize sources into a cohesive narrative. Ground all nontrivial claims with `<cite id="...">...</cite>`.

CRITICAL: formatting instructions appended to the user's prompt dictate the shape of your answer, but your dynamically updated `<rubric>` dictates the substance.

PHASE 1: PLANNING & INITIATION

STEP 0: Exploratory Thinking

Begin with a `<think>` block. Use this as a computational workspace for brief, unstructured brainstorming. Identify initial questions, obvious roadblocks, and missing variables.

STEP 1: The Structured Plan

Output a single `<structured_plan>` block containing exactly three sections, building upon each other logically:

1. `<deep_analysis>`: Discover the True Intent.
 - Complexity Assessment: Evaluate the multi-dimensional complexity (retrieval, reasoning, insight, formulation).
 - Explicit Needs: What they are directly asking for.
 - Implicit Needs & Gaps: Hidden constraints, missing variables, or potential roadblocks.
2. `<rubric>`: Define the Strict Grading Criteria. Act as an expert grader creating a rigorous checklist. Draw upon the `<deep_analysis>` as a foundation. DO NOT focus on formatting; focus on required content, intellectual depth, and logical constraints.
 - Knowledge Checklist: The exact facts, definitions, comparisons, or data points required.
 - Analytical & Synthesis Criteria (Optional, for complex queries): What intellectual connections must the response achieve?
 - Negative Constraints (Pitfalls): What must the response explicitly AVOID?

3. <research_plan>: Formulate the Strategy. Create a logical roadmap to satisfy the rubric. Simple queries need a linear step or two; complex queries require a conditional, look-ahead strategy.

STEP 2: The First Tool Call

Immediately after closing </structured_plan>, execute ONLY the first step of your research plan. Under NO circumstances should you output an <answer> in this first turn.

PHASE 2: SEARCH & SYNTHESIS

STEP 0: Evaluation Thinking

Begin with a <think> block for raw inner monologue to digest new evidence.

STEP 1: State Evaluation & Action

Output a <state_evaluation> block to analyze the tool output, then choose exactly ONE path:

PATH A: Continue Research (More Information Needed)

- Direct Search: If the current plan and rubric are still valid, output your next <call_tool>.
- Update & Search: If new information invalidates initial assumptions, output an updated <structured_plan> with revised <deep_analysis>, <rubric>, and/or <research_plan>, then output your next <call_tool>.

PATH B: Ready to Answer (Evidence Sufficient)

1. The Review: Output a <review> block containing:
 - <rubric_review>: Systematically map retrieved evidence back to the Phase 1 <rubric>. Verify knowledge checklist items. Re-articulate synthesis criteria and negative constraints in the context of retrieved evidence.
 - <writing_plan>: Outline the final answer's architecture. For complex queries: define a unified core thesis, value proposition, narrative architecture, and citation mapping.
2. The Final Answer: Output the response within <answer>...</answer> tags.

Available Tools

- google_search: Powered by a grounded AI reasoning engine. Use direct, highly specific search queries.
Input: <call_tool name="google_search">query</call_tool>
- snippet_search: Focused retrieval from scientific papers.
Input: <call_tool name="snippet_search">query</call_tool>
Optional parameters: limit, year, fieldsOfStudy.

Tool Output Format

```

- Results are appended in <tool_output>...</tool_output> tags.
- For google_search: summary text with grounding snippets.
- For snippet_search: <snippet id="ID">content</snippet>.

## WORKFLOW EXAMPLE
[... A complete worked example demonstrating the full scaffold
on a sample query, showing Phase 1 (think, structured_plan,
first call_tool), Phase 2 iterations (state_evaluation,
continued search, dynamic plan updates), and the review and
answer synthesis. Omitted for brevity. ...]

## CRITICAL CONSTRAINTS
- Mandatory Starting Tag: Output MUST begin with <think>.
- Mandatory First Search: First turn MUST end with
  </call_tool>. Never answer without searching.
- One Action Per Turn: End with </call_tool> OR </answer>.
  STOP generating immediately after. Do NOT simulate
  <tool_output> yourself.
- Mandatory Citations: Wrap claims in
  <cite id="S_123">claim text</cite>. Never use empty tags.
  Multiple sources: <cite id="S_1, S_2">claim</cite>.
- Substance over Syntax: The <rubric> dictates substance;
  formatting instructions dictate only surface-level shape.
    
```

Additional instructions. Depending on the expected response format, one of three task-specific instructions is appended to the user’s query:

- **Exact answer:** “Search iteratively to find the precise answer. Format: <answer>\boxed{exact answer}</answer>.”
- **Short form:** “Search iteratively to gather evidence from multiple credible sources. Synthesize into a short paragraph within <answer>...</answer> tags.”
- **Long form:** “Search iteratively to gather evidence from multiple credible sources. Synthesize into a comprehensive, evidence-backed long-form response within <answer>...</answer> tags.”

B.2. SFT Data Generation Process

To instill the rubric-guided scaffold into the base Qwen3-8B model, we perform supervised fine-tuning using teacher-generated trajectories. The data generation process consists of a multi-round, tool-augmented pipeline that produces complete stage-structured trajectories from a teacher model.

B.2.1. Teacher Model and Prompt Adaptation

We use Gemini-3.1-Pro as the teacher model. The training queries are drawn from the same dataset used by Dr. Tulu (Shao et al., 2025), which contains ~13K diverse, open-ended research questions spanning multiple domains. Each query is paired with a response format label (exact_answer, short_form, or long_form) that determines the additional instruction appended to the prompt.

A key challenge is that the SFT data generation prompt must be adapted for Gemini, which differs from the target Qwen3-8B model in several important ways.

Reasoning traces and <scratchpad>. Our scaffold includes unstructured <think> blocks at the start of each turn to preserve the base Qwen3-8B policy’s post-training reasoning behavior. However, directly prompting Gemini to produce <think> tags is blocked by the API, and internal thinking traces are inaccessible due to anti-distillation policies. We use <scratchpad> as a substitute reasoning tag during SFT data generation; these are converted to <think> in post-processing. While these traces do not capture Gemini’s true internal chain-of-thought, they provide visible planning rationale and evidence assessment that teaches the student model the habit of explicit reasoning before acting, without disrupting its native <think> patterns.

Stagewise prompt separation. We separate the system prompt into a first-round variant (containing only Phase 1 instructions) and a later-rounds variant (containing only Phase 2 instructions), since each round is a separate Gemini API call. This separation serves two purposes: it keeps each prompt focused on the current stage’s requirements (reducing prompt length and improving adherence), and it ensures the model invests heavily in planning at step 1 (generating the full <structured_plan> with analysis, rubric, and research plan) rather than skipping ahead to search or answering directly. Each variant explicitly constrains the turn-ending tag: the first round must end with </call_tool>, and later rounds must end with either </call_tool> or </answer>.

We present the core of each generation prompt below (omitting tool documentation and workflow examples, which mirror the final agent prompt).

SFT Generation Prompt: First Round (Phase 1 Only)

```
# Role: Elite Research Planning & Initiation Agent

You are an elite Research Planning Agent. Your purpose is to
analyze complex queries, generate a rigorous, visible research
plan, and initiate the very first search query to kick off the
research process. You DO NOT answer the user’s question
directly, and you DO NOT simulate tool outputs.

## CORE PRINCIPLE: ADAPTIVE COGNITIVE EFFORT
[... same as final prompt ...]

## Process

STEP 0: Exploratory Scratchpad
Begin with a <scratchpad> block. This is your mandatory
computational workspace. Use this space for a brief,
unstructured exploration of the user’s query --- identify
initial hurdles and missing variables before moving to
structured output. Do not output XML tags inside scratchpad.

STEP 1: The Structured Plan
Output a single <structured_plan> block containing exactly
three sections, building upon each other logically:

1. <deep_analysis>: Discover the True Intent.
  - Complexity Assessment: Evaluate the multi-dimensional
    complexity (retrieval, reasoning, insight, formulation).
  - Explicit Needs: What they are directly asking for
    (including structural/formatting instructions).
```


- Implicit Needs & Gaps: Hidden constraints, missing variables, or potential roadblocks.
2. <rubric>: Define the Strict Grading Criteria. Act as an expert grader creating a rigorous checklist. DO NOT focus on formatting; focus on content, depth, and constraints.
 - Knowledge Checklist: Exact facts, definitions, data points required (e.g., "The response explicitly defines the von Neumann bottleneck").
 - Analytical & Synthesis Criteria (Optional, for complex queries): What intellectual connections must the response achieve?
 - Negative Constraints (Pitfalls): What must the response explicitly AVOID? (e.g., "The response avoids presenting industry blogs as academic consensus").
 3. <research_plan>: Formulate the Strategy. Create a logical roadmap to satisfy the rubric. Simple queries: linear steps. Complex queries: conditional, look-ahead strategy (e.g., "Step 1: Find X. Step 2: If X confirms Y, investigate Z; if inconclusive, fallback to W").

STEP 2: The First Tool Call

Immediately after closing </structured_plan>, execute ONLY the first step of your <research_plan>. Once you output </call_tool>, you must STOP generating text.

CRITICAL CONSTRAINTS

- Output MUST begin with <scratchpad>, then <structured_plan>.
- Execute EXACTLY ONE tool call.
- DO NOT simulate <tool_output>.
- DO NOT attempt to write the final <answer>.

[... tool documentation and workflow examples omitted ...]

SFT Generation Prompt: Later Rounds (Phase 2 Only)

Role: Elite Iterative Research & Synthesis Agent

You are in the active research phase. You have already generated an initial Rubric and Research Plan. Your goal is to evaluate the latest retrieved information, dynamically adjust your strategy if necessary, and either continue searching or generate the final answer.

CONTEXT: THE PHASE 1 FOUNDATION

Prior to your current phase, an initial <structured_plan> was generated. This plan is your foundational blueprint consisting of <deep_analysis>, <rubric>, and <research_plan>. You must constantly evaluate incoming evidence against this baseline.

Process & Decision Tree

STEP 0: Evaluation Scratchpad

Begin with a <scratchpad> block. This is your mandatory computational workspace for raw, unstructured inner monologue to digest newly retrieved evidence. If data contradicts assumptions, briefly identify the conflict and note your pivot. If evidence is sufficient for a complex query, pinpoint writing hurdles before resolving them in <review>.

STEP 1: State Evaluation & Action

Output a <state_evaluation> block to formally analyze the latest tool output, then choose exactly ONE path:

PATH A: Continue Research (More Information Needed)

- Direct Search: If the current plan and rubric are still valid, output your next <call_tool> immediately.
- Update & Search: If new information invalidates initial assumptions, triggers a fallback, or reveals a deeper multi-hop requirement:
 1. Output an updated <structured_plan> with revised <deep_analysis>, <rubric>, and/or <research_plan>. (Only include sections that require updates.)
 2. Then output your next <call_tool>.

PATH B: Ready to Answer (Evidence Sufficient)

1. The Review: Output a <review> block containing:
 - <rubric_review>: Systematically map retrieved evidence back to the Phase 1 <rubric> (plus any dynamic updates).
 - * Knowledge Verification: List the critical facts and data points that satisfy the Knowledge Checklist.
 - * Synthesis & Constraints Restatement: Re-articulate Analytical Criteria and Negative Constraints in the context of retrieved evidence. Transform abstract rubric targets into concrete directives for drafting.
 - <writing_plan>: Outline the final answer architecture, scaling effort to query complexity:
 - * Low-Complexity: State verified facts directly.
 - * High-Complexity: Define (1) unified core thesis, (2) value proposition, (3) narrative architecture with section-by-section outline, (4) citation mapping specifying how verified facts integrate into sections.
2. The Final Answer: Output within <answer>...</answer> tags. Depth and formatting MUST match the writing plan. Follow the Knowledge Checklist and honor all constraints from the <rubric_review>.

CRITICAL CONSTRAINTS

- Output MUST begin with <scratchpad>, then <state_evaluation>.
- One Action Per Turn: end with </call_tool> OR </answer>.
- STOP generating immediately after the closing tag.
- Mandatory Citations: wrap claims in <cite id="S_123">claim text</cite>. Never use empty tags.
- Substance over Syntax: formatting instructions dictate shape; the <rubric> dictates substance. Enforce the rubric before drafting.

[... tool documentation and workflow examples omitted ...]

The key difference from the unified final agent prompt is the strict separation of responsibilities: the first-round prompt forbids answering and requires a full planning phase, while the later-rounds prompt assumes the plan already exists and focuses on evidence evaluation and synthesis. This separation ensures the teacher invests in thorough planning rather than short-circuiting to an answer.

Common failure modes. Despite these adaptations, Gemini-3.1-Pro frequently violates the scaffold constraints. Common failures include answering directly from internal knowledge without searching, producing output that does not end with the required closing tag, omitting required structural elements such as `<rubric>` or `<state_evaluation>`, and generating variant tool names that do not match the expected schema. These violations necessitate aggressive rejection sampling (described below).

B.2.2. Multi-Round Generation Pipeline

Because the scaffold requires iterative tool interaction, we cannot generate complete trajectories in a single model call. Instead, we use a multi-round pipeline that alternates between model generation and real tool execution:

1. **Round 1 (Initiation):** The teacher receives the first-round system prompt and user query. It generates Phase 1 output: `<scratchpad>`, `<structured_plan>` (with `<deep_analysis>`, `<rubric>`, `<research_plan>`), and the first `<call_tool>`. Generation stops at the closing `</call_tool>` tag.
2. **Tool routing and execution:** Each `<call_tool>` is parsed to extract the tool name and query. The pipeline routes calls to one of two backends:
 - `google_search`: Submitted as Vertex AI batch prediction jobs using Gemini with Google Search grounding enabled, which returns AI-synthesized summaries with grounding snippets.
 - `snippet_search`: Executed asynchronously against the Semantic Scholar API, which returns paper excerpts matching the query.
 Both backends run in parallel to maximize throughput. Responses where the tool fails to call a valid tool (e.g., answering from internal knowledge) are separated into an error stream for potential reprocessing.
3. **Rounds 2–N (Research iteration):** The tool output is appended to the conversation history, and the teacher receives the later-rounds system prompt. It generates the next Phase 2 turn: `<scratchpad>`, `<state_evaluation>`, and either another `<call_tool>` or the `<review>` and `<answer>`. This loop continues until the teacher produces a closing `</answer>` tag or a maximum of 10 rounds is reached.
4. **Completion detection:** A trajectory is considered complete when the teacher’s output ends with `</answer>`. Incomplete trajectories that exhaust the maximum rounds are discarded. Because Gemini often fails to produce a well-formed closing tag, a substantial fraction of trajectories (~15–25%) are discarded at this stage.

B.2.3. Quality Filtering and Rejection Sampling

Raw teacher trajectories undergo rejection sampling and several filtering steps. This is critical because Gemini-3.1-Pro, despite being a strong model, frequently violates the scaffold’s structural constraints when generating multi-round tool-augmented trajectories.

Rejection criteria. The following trajectories are rejected:

- **Missing `</answer>` tag (hard reject):** The most common failure mode. The teacher generates extensive content but never produces the closing `</answer>` tag, either because it runs out of tokens, enters an endless research loop, or simply stops mid-sentence. These trajectories are discarded entirely.
- **No valid tool call in non-final rounds:** Each non-final round must end with a valid `<call_tool>` tag containing a parseable tool name and query. Trajectories where the model skips tool use and attempts to answer from internal knowledge (e.g., producing a long response without any `<call_tool>`) are rejected. This is the second most common failure mode, particularly on simple factual queries where Gemini “knows” the answer and refuses to search.
- **Missing structural elements:** Trajectories must contain valid `<structured_plan>` (with `<deep_analysis>` and `<rubric>`) in the first round, and `<state_evaluation>` in subsequent rounds. Those with malformed XML or missing required sections are discarded.
- **Consecutive tool errors:** Trajectories with two or more consecutive rounds where the tool backend returned an error (e.g., search blocked by safety filters, API rate limits, “no grounding data available”) are discarded. Single error rounds followed by successful recovery are retained, as these teach the model error-resilient behavior.

Post-processing transformations. Accepted trajectories undergo the following conversions:

- **Reasoning tag conversion:** All `<scratchpad>...</scratchpad>` tags from Gemini’s output are converted to `<think>...</think>` for compatibility with Qwen3’s chat template, which uses `<think>` as the native reasoning tag.
- **Tool name normalization:** Gemini occasionally produces variant tool names (e.g., `google_web_search`, `Scholar_Search`) or malformed attribute strings. These are normalized to the canonical `google_search` and `snippet_search` names.

B.2.4. Training Data Format

Each filtered trajectory is converted into a single-turn ChatML conversation with three messages:

- **System:** The full agent system prompt (the unified Qwen3 version from Sec. B.1, not the Gemini-adapted generation prompt).
- **User:** The original query with the format-specific additional instruction appended.
- **Assistant:** The complete multi-round trajectory, with tool outputs wrapped in special span-masking markers. Only the model-generated tokens (thinking, planning, tool calls, state evaluation, review, and answer) receive gradient during training; tool output tokens are masked so the model does not learn to memorize search results.

Finally, this process yields around 11k SFT samples, approximately 2k fewer than DR Tulu due to the repeated errors described above, which are filtered out.

C. Details on Stagewise Evolving Rubric Evaluation

This section provides implementation details for the stagewise evolving rubric design described in Section 3.3. The judge operates in two phases: *adaptive rubric generation* proposes new rubrics by comparing trajectories for the same question, and *stagewise scoring* evaluates each trajectory against the current rubric set. Both phases use an LLM judge (Gemini-3-Flash) with structured JSON output.

C.1. Rubric Buffer

For each training question, we maintain a *rubric buffer* with two types of rubrics: *persistent rubrics* (static, ground-truth rubrics from the training data, never removed) and *active rubrics* (adaptive rubrics proposed online by the judge, organized by stage and subject to per-stage capacity caps). The persistent rubrics are adapted from Shao et al. (2025) and only used for the final answer stage. At each training step, for each unique question in the batch, the judge (i) generates new adaptive rubrics by comparing the rollout group, (ii) adds them to the buffer, (iii) scores all trajectories against the combined rubric set, and (iv) removes low-discrimination rubrics that did not meaningfully separate trajectory quality.

C.2. Adaptive Rubric Generation

The rubric generation prompt instructs the judge to identify the most discriminative, stage-local criteria that explain why some trajectories are better than others. Several design principles considered:

- **Discriminative and specific:** Each rubric must actually separate stronger from weaker trajectories for the same question. Descriptions must reference concrete aspects of the question so a separate scorer can unambiguously judge them. Vague rubrics like “good research quality” are prohibited.
- **Stage-local and non-redundant:** Rubrics target each stage’s specific responsibility. The judge avoids creating positive and negative mirror versions of the same criterion, and skips criteria already covered by existing rubrics.
- **Anti-hack:** The judge must not create rubrics that merely check XML tag existence, reward formal obedience to a weak plan, overfit to one trajectory’s wording, or confuse self-consistency with true quality.

A key feature enabled by our rubric-guided scaffold is that the judge can use the agent’s own `<rubric>` blocks from different trajectories as *references* when proposing adaptive rubrics—strong rubrics across trajectories can reveal important task dimensions and failure modes. However, a trajectory’s own rubric is never the scoring standard: the judge is instructed to never reward following a weak self-rubric or penalize evidence-based improvement beyond the original plan.

Rather than prescribing fixed templates, the prompt provides stage-specific guidance as starting points. For each stage, a central question frames the evaluation: “Did the agent understand the problem?” (Plan), “Did it search effectively and adapt?” (Research), “Did it honestly audit readiness?” (Review), and “Is the final answer high-quality?” (Answer). Common useful dimensions are suggested (e.g., query specificity, evidence-based pivoting for Research), but the judge is encouraged to discover what matters for each specific question.

Each rubric item contains a `title`, `description`, and `weight` (1=minor, 2=important, 3=critical), organized into positive and negative rubrics per stage. Target counts are 1–4 rubrics per stage for stages 1–3 and 2–5 for stage 4, treated as targets rather than quotas. The full rubric generation prompt is shown below:

Stagewise Adaptive Rubric Generation Prompt (Core)

You are an expert evaluator generating stagewise comparative rubrics for multiple full trajectories produced by the SAME research agent on the SAME question.

Goal

Identify the most discriminative, stage-local criteria that

explain why some trajectories are better than others. Evaluate real process quality and answer quality - not XML compliance, verbosity, or shallow self-consistency.

How to Read the Trajectories

Each trajectory is a structured full execution trace:

- Stage 1 - Plan: <structured_plan> with <deep_analysis>, <rubric>, <research_plan>
- Stage 2 - Research: iterative <state_evaluation>, tool calls, and possible plan/rubric updates
- Stage 3 - Review: <review> with <rubric_review> and <writing_plan>
- Stage 4 - Answer: final <answer>

Do not reward a stage just because the relevant tags exist. Judge whether that stage actually performs its intended function well.

How to Use Agent-Generated Rubrics

A trajectory's own Stage 1 <rubric> is never the scoring standard. Use it only as a reference signal:

- it shows what the agent recognized as important,
- it may surface useful task dimensions,
- it may expose shallow planning if it misses critical needs.

But: never reward following a weak self-rubric, never let a narrow self-rubric lower the real standard, never penalize evidence-based improvement beyond the original plan.

Stage-by-Stage Guidance

For each stage, the most discriminative criteria often emerge from the specific question and trajectories themselves. The dimensions below are common starting points:

- Stage 1 - PLAN: task understanding depth, identification of hidden constraints, rubric specificity, plan proportionality.
- Stage 2 - RESEARCH: query specificity, evidence-based pivoting, triangulation, stopping criteria.
- Stage 3 - REVIEW: honest gap assessment, evidence-to-structure mapping, drafting plan quality.
- Stage 4 - ANSWER: readability, instruction following, insightfulness, comprehensiveness, correctness, evidence grounding. Evaluated independently from earlier trajectory.

Anti-Hack Rules

Do NOT create rubrics that: only check tag existence, reward formal obedience to a bad plan, overfit to one trajectory's wording, confuse self-consistency with true quality, punish evidence-based improvement, or reward unnecessary overhead.

Selection Rules

- Fewer, sharper rubrics over many generic ones.
- Skip criteria where all trajectories perform similarly.
- Never create positive and negative mirrors of the same idea.
- Conservative negative rubrics: target clear failure modes.


```
## Output Format
Return rubrics by stage (stage_1 through stage_4), each with
positive_rubrics and negative_rubrics. Each item: title,
description, weight (1-3).
```

C.3. Stagewise Scoring

After rubric generation, each trajectory is scored independently against the combined rubric set (persistent + active). Positive rubrics use a 0/1/2 scale (absent / partial / full), and negative rubrics use the same scale but inverted during aggregation: a score of 0 (no flaw) contributes positively. Each evaluation includes a brief justification grounded in specific trajectory evidence. For each stage k , the per-stage score $R_{i,k} \in [0, 1]$ is a weighted average of rubric scores within that stage, with negative scores inverted before aggregation.

Stagewise Scoring Prompt

You are an expert evaluator. You will receive an AI research agent’s full trajectory and a set of rubrics grouped by stage. Score each rubric strictly based on the trajectory evidence.

Trajectory Structure

- Stage 1 (Plan): start through end of `</structured_plan>`
- Stage 2 (Research): after `</structured_plan>` through all tool-call loops
- Stage 3 (Review): last `<think>` before `<review>` through `</review>`
- Stage 4 (Answer): `<answer>` through end of text

Scoring Rules

[POSITIVE] rubrics:

- 2 = FULLY exhibits this quality
- 1 = PARTIALLY exhibits it
- 0 = Completely ABSENT or fails

[NEGATIVE] rubrics:

- 2 = Fully EXHIBITS the flaw
- 1 = PARTIALLY exhibits the flaw
- 0 = Successfully AVOIDS the flaw

Instructions

- Score each rubric independently.
- Use the rubric description as the sole scoring criterion.
- Provide brief justification grounded in specific trajectory evidence (quote or reference concrete content).
- Return the exact rubric index with each score.

C.4. Buffer Management and Implementation

Buffer dynamics. Each stage maintains a capacity cap on active rubrics (3, 2, 2, 3 for stages 1–4 in our experiments). When a stage exceeds its cap after scoring, rubrics with the lowest discrimination (measured by score variance across the rollout group) are removed. Persistent rubrics are never removed. When a question reappears in a later training step, new adaptive rubrics from the improved

policy are added, potentially replacing stale ones that no longer discriminate. This ensures the judge’s criteria co-evolve with the policy, consistent with the evolving rubric paradigm of [Shao et al. \(2025\)](#), extended here from answer-only to stagewise evaluation.

Structured JSON output. Both rubric generation and scoring use Gemini’s structured output mode (`response_mime_type="application/json"`) with Pydantic-derived JSON schemas. This guarantees schema compliance (rubrics organized by stage with title/description/weight, scores with indices and justifications) without fragile post-hoc text parsing.

Batch parallelism. At each training step, the batch contains $32 \text{ unique questions} \times 8 \text{ rollouts} = 256$ trajectories. All 32 rubric generation calls are launched concurrently via `asyncio.gather`, followed by all 256 scoring calls in parallel. Each call includes exponential-backoff retry logic (up to 5 retries). Failed calls are treated as “no new rubrics” (generation) or fall back to terminal answer score only (scoring).

Efficiency. Rubric generation requires only 32 calls per step (one per question, not per trajectory). Scoring requires 256 calls but each is lightweight ($\sim 8\text{--}15$ rubrics per trajectory). The combined rubric generation + scoring phase takes approximately 5 minutes per step, overlapping with the training engine’s gradient computation. The judge model (Gemini-3-Flash) is chosen for cost and latency; the structured output constraint and detailed prompt compensate for the smaller model’s limitations.

D. Asynchronous Reflection Pipeline and Windowed Curriculum

This appendix provides the implementation details for the meta-policy training pipeline and the windowed retrieval curriculum described in Section 3.4.

D.1. Training Pipeline Architecture

Each RL training step in our agentic loop involves several sequential phases: rollout generation (multi-turn tool-augmented trajectories via vLLM), stagewise judge scoring (SS-GRPO reward computation), and policy gradient update. Reflection generation (prompting the backbone to produce rubric-grounded reflections from sampled trajectories) introduces an additional compute requirement that we overlap with the main loop.

Our pipeline uses three concurrent threads:

1. **Main thread (training engine):** Performs gradient updates on the policy. Alternates between Phase A (meta-policy update on deferred reflections from the previous step) and Phase B (task-policy SS-GRPO update on the current step’s rollouts).
2. **Inference thread (vLLM):** Generates multi-turn tool-augmented rollouts. During Phase A, the inference engine begins generating the next batch of rollouts while the training engine trains on deferred reflections—fully overlapping inference and reflection training.
3. **Data preparation thread:** Runs judge scoring (async Gemini API calls), launches reflection generation and reflection judging as concurrent background tasks, and prepares packed training batches. Reward scoring results are returned immediately for task-policy training; reflection scoring continues asynchronously.

D.2. One-Step Deferred Reflection Training

A synchronous implementation would block the next task rollout until reflection generation, judging, and the meta-policy update are finished. We instead defer reflection training by one RL step, as illustrated in the bottom panel of Fig. 4.

At step N :

1. The inference engine generates task rollouts for step N .
2. The data preparation thread scores step N rollouts (SS-GRPO rewards) and simultaneously launches background tasks for: (a) reflection rollout generation via vLLM, producing n candidates per sampled trajectory, and (b) judge scoring of the reflection candidates.
3. Reward results are returned immediately; the training engine performs the SS-GRPO task-policy update on step N .
4. Meanwhile, reflection scoring completes in the background. Accepted reflections are inserted into the rubric bank, and the scored reflection samples are placed in a *deferred buffer*.
5. At the start of step $N+1$, during **Phase A**, the training engine trains on the deferred reflection buffer from step N *while* the inference engine generates step $N+1$ rollouts.

This one-step staleness trades exact synchrony for higher infrastructure utilization: both inference and training engines remain continuously occupied, and meta-policy training adds effectively no extra wall-clock overhead to the SS-GRPO loop.

D.3. Windowed Curriculum

Since reflection training is deferred by one step and bank insertion happens asynchronously, a naïve curriculum that immediately revisits a query after its first encounter may attempt within-episode retrieval before the corresponding reflection has been generated and accepted. To avoid this conflict, we introduce a windowed curriculum with window size K that provides sufficient temporal separation between a query’s first and second encounters.

Training alternates between two phases within each window of $2K$ steps:

1. **New-query phase** (steps $1, \dots, K$ within the window): The dataloader samples K fresh query batches $\{B_1, B_2, \dots, B_K\}$. For each batch, the bank performs *cross-episode retrieval*: semantically similar items from past (different) queries are injected as few-shot exemplars. At the end of each step, reflection generation is launched asynchronously in the background.
2. **Repeat phase** (steps $K+1, \dots, 2K$ within the window): The same batches are replayed in order: $B_1, B_2, \dots, B_K$. For each batch, the bank performs *within-episode retrieval*: the exact reflection generated during the new-query phase is retrieved and injected as direct self-guidance.

The K -step gap between a query’s first encounter and its repeat guarantees that the deferred reflection pipeline (generation, judging, bank insertion, and at least one deferred training step) has fully completed before within-episode retrieval is attempted. In our experiments, we use $K=3$, providing a comfortable margin over the one-step deferral.

D.4. Trajectory Sampling and Candidate Generation

For each query in the new-query phase, we randomly sample one trajectory from the rollout group for reflection generation. The sampling is uniform random over trajectory indices, without length bias or score-based selection, mirroring inference conditions where the agent does not have access to

Step in window	Phase	Data	Retrieval mode
1	New	B_1 (fresh)	Cross-episode (similar)
2	New	B_2 (fresh)	Cross-episode (similar)
3	New	B_3 (fresh)	Cross-episode (similar)
4	Repeat	B_1 (replay)	Within-episode (exact)
5	Repeat	B_2 (replay)	Within-episode (exact)
6	Repeat	B_3 (replay)	Within-episode (exact)

Figure 7 | Windowed curriculum with $K=3$. Each window of $2K=6$ steps alternates between new queries with cross-episode retrieval and repeated queries with within-episode exact retrieval. The 3-step gap ensures the deferred reflection pipeline has fully completed before the repeat.

privileged scoring information. From the sampled trajectory, we generate $n=8$ reflection candidates via vLLM parallel sampling (temperature 0.7, single prompt, n completions). Each candidate is independently scored by the judge, and only the highest-scored candidate with valid output format is accepted into the bank. Each query’s reflection generation is independent, so a failure for one query does not affect others.

D.5. Bank Persistence and Retrieval

The rubric bank maintains an in-memory FAISS index over query embeddings (computed via Qwen3-Embedding-0.6B on CPU). For cross-episode retrieval, the query embedding is compared against stored embeddings via inner-product similarity, and the top- k items (default: $k=2$) are returned. For within-episode retrieval, items are matched by exact question hash (SHA-256). When a query already has a bank item and the same query is encountered in a later training step, the new reflection overwrites the old one, keeping the bank synchronized with the evolving policy.

The bank is persisted every 10 steps and at every model checkpoint, using atomic writes (write to temp file, then rename) to prevent corruption. On training resume, the bank is restored from the checkpoint corresponding to the same global step as the model weights, ensuring consistency between the policy parameters and the memory contents.

D.6. Prompt Templates

D.6.1. Reflection Generation Prompt

The policy model receives the following system prompt when generating rubric-grounded reflections:

Reflection System Prompt

You are a strict postmortem editor for long-form deep research QA.

You will receive:

- a question
- a research trajectory ending in a final
<answer>...</answer>

Your job is not to answer the question again. Your job is to extract the most useful guidance for:

1. a stronger next attempt on this question, and

2. similar long-form research questions.

First, read the final <answer>.

Judge it on: correctness and calibration;
instruction-following and coverage; research quality
and verification; synthesis and insight;
communication and structure.

Then identify the 2-4 highest-leverage strengths or
weaknesses, trace them back to the visible trajectory,
and turn them into reusable guidance.

Rules:

- Stay grounded in the provided question and trajectory.
- Do not invent flaws just to be critical.
- Do not speculate about hidden reasoning not shown in the trajectory.
- Do not rewrite the answer.
- Do not give generic advice like "be more careful."
- Write specific guidance that is grounded in this case but transferable to similar questions.

Output format (exactly two blocks):

```
<reflection_rubrics>
Critical Requirement / Analytical Approach
/ Communication & Structure / Pitfalls
</reflection_rubrics>
```

```
<reflection_takeaways>
1-4 concise portable lessons
</reflection_takeaways>
```

The user message provides the question and the full research trajectory, instructing the model to examine the final answer, trace strengths and weaknesses, and produce both output blocks.

D.6.2. Judge Scoring Prompt

An LLM judge (Gemini) evaluates each reflection candidate with privileged access to the graded trajectory, including per-rubric scores and evaluator justifications. The judge scores each candidate on how useful it would be for future attempts:

Reflection Judge System Prompt (abridged)

You are an expert judge evaluating reflection candidates produced by a research agent.

Scoring Dimensions (each 0.0 to 1.0):

Diagnostic Accuracy (weight 0.4)

- Does it correctly identify the main strengths and weaknesses? Is it aligned with the evaluator

justifications and visible trajectory?
 Penalize: missing a major failure clearly flagged by the evaluator; inventing problems not supported by the trajectory; misidentifying root causes.

Specificity (weight 0.3)

- Is the guidance concrete, actionable, and tailored for next attempt of the question?
 Penalize: generic advice ("be more thorough"); checklist items that could apply to any task; merely restating evaluator language.

Scope & Balance (weight 0.3)

- Is the guidance helpful for other questions?
 Are takeaways complementary?
 Penalize: missing an important category; repeating the same point across sections.

Rules:

- Score based on the reflection's quality as future guidance, not on the trajectory's quality.
- A reflection on a perfect trajectory can score low if it is generic.
- A reflection on a failed trajectory can score high if it precisely identifies root causes.

The user message provides the original question, all reflection candidates, and the graded trajectory with evaluator feedback. The judge returns per-candidate scores across all three dimensions.

D.6.3. Injection Formats

Retrieved bank items are injected as a `<reference_examples>` block appended to the user message before tokenization. The two retrieval modes from Section 3.4—within-episode refinement and cross-episode transfer—use distinct preambles to condition the agent's use of the retrieved guidance.

Within-episode injection. When the agent revisits a question it has attempted before (repeat phase of the windowed curriculum), the accepted reflection from the previous encounter is injected:

Within-Episode Injection Format

```
<reference_examples>
You have attempted this exact question before. Below
are the rubrics and reflections from your previous
attempt. Use them to improve your planning and
answering -- avoid past mistakes and build on what
worked.

## Your Previous Attempt on This Question:

### Rubrics:
{rubrics from bank item}
```

```

### Takeaways:
{reflections from bank item}
</reference_examples>

{Original user question}

```

Cross-episode injection. For a new query (new-query phase), the bank retrieves the top- k semantically similar items and presents them as rubric-grounded exemplars:

Cross-Episode Injection Format

```

<reference_examples>
Below are rubrics and reflections from similar
questions that were previously analyzed. Use them as
reference to guide your planning and answering --
adapt to the current question's specific needs.
Do NOT copy them verbatim; extract what is relevant
and adjust.

## Similar Question 1:
{question from bank item 1}

### Rubrics:
{rubrics from bank item 1}

### Takeaways:
{reflections from bank item 1}

---

## Similar Question 2:
{question from bank item 2}

### Rubrics:
{rubrics from bank item 2}

### Takeaways:
{reflections from bank item 2}
</reference_examples>

{Original user question}

```

In both cases, the reference block precedes the original user question, so the agent sees the guidance before beginning its structured plan. The within-episode format signals direct self-improvement (“avoid past mistakes”), while the cross-episode format signals analogical transfer (“adapt to the current question’s specific needs”).

E. Theoretical Analysis

E.1. Value of Stage Information

In the main text, Theorem 1 is stated using the informal notation h, c, z inside conditional expectations for readability. In this appendix, we make the probability space explicit: H, C, Z denote random variables, while h, c, z denote realized values. We also slightly sharpen the strictness condition in the theorem by requiring the compared stages to have positive conditional probability given c ; without this requirement, the pointwise quantities $\mathbb{E}[U(H, a) \mid C = c, Z = z]$ on impossible stage events are version-dependent.

The result below is a value-of-information statement specialized to structured agent reasoning. It compares two information structures, $\sigma(C)$ and $\sigma(C, Z)$. Unlike full state-abstraction theorems, it does *not* assume that (C, Z) is sufficient for the entire history H ; it only quantifies the gain from making the stage label explicit under a compressed decision state.

Assumption 1 (Setup for Theorem 1). *Throughout this subsection, the following assumptions hold.*

1. **Underlying probability space.** *There exists a probability space $(\Omega, \mathcal{F}, \mathbb{P})$ and a random decision history*

$$H : \Omega \rightarrow \mathcal{H},$$

where $(\mathcal{H}, \mathcal{B}(\mathcal{H}))$ is a measurable space. The law of H is denoted by $d := \mathbb{P} \circ H^{-1}$. We interpret H as a random reachable decision point sampled from some fixed distribution over histories.

2. **Finite action space.** *The action space $\mathcal{A}$ is a finite nonempty set. Consequently, every maximization and argmax over $\mathcal{A}$ is well-defined and nonempty.*
3. **Compressed context and stage label.** *There exists a measurable map*

$$\phi : \mathcal{H} \rightarrow C$$

into a standard Borel space $(C, \mathcal{B}(C))$, and a measurable stage map

$$\psi : \mathcal{H} \rightarrow [K] := \{1, \dots, K\},$$

such that

$$C := \phi(H), \quad Z := \psi(H).$$

Since C is standard Borel and $[K]$ is finite, the regular conditional objects used below exist.

4. **Integrable utility.** *For each $a \in \mathcal{A}$, there exists a measurable utility function*

$$U(\cdot, a) : \mathcal{H} \rightarrow \mathbb{R}$$

such that

$$\mathbb{E}[|U(H, a)|] < \infty.$$

In our application, $U(H, a)$ can be instantiated as a continuation utility or continuation value associated with choosing action a at decision point H .

Definition 1 (Conditional mean utilities and stage probabilities). *Adopt Assumption 1. For each $a \in \mathcal{A}$, fix measurable versions*

$$\bar{q}(\cdot, a) : C \rightarrow \mathbb{R}, \quad q(\cdot, \cdot, a) : C \times [K] \rightarrow \mathbb{R}$$

such that

$$\bar{q}(C, a) = \mathbb{E}[U(H, a) \mid \sigma(C)] \quad \text{almost surely,}$$

and

$$q(C, Z, a) = \mathbb{E}[U(H, a) \mid \sigma(C, Z)] \quad \text{almost surely.}$$

For each $z \in [K]$, fix a measurable version

$$p(z \mid \cdot) : C \rightarrow [0, 1]$$

such that

$$p(z \mid C) = \mathbb{P}(Z = z \mid \sigma(C)) \quad \text{almost surely.}$$

We write

$$\mathbb{P}_C := \mathbb{P} \circ C^{-1}$$

for the law of C , and define the stage support at a context $c \in C$ by

$$S(c) := \{z \in [K] : p(z \mid c) > 0\}.$$

Remark 1 (Well-definedness). Because $\mathcal{A}$ is finite and each $U(H, a)$ is integrable, the random variables $\bar{q}(C, a)$ and $q(C, Z, a)$ are integrable for every $a \in \mathcal{A}$. Indeed, by Jensen's inequality for conditional expectations,

$$|\bar{q}(C, a)| = |\mathbb{E}[U(H, a) \mid \sigma(C)]| \leq \mathbb{E}[|U(H, a)| \mid \sigma(C)] \quad \text{a.s.,}$$

and therefore

$$\mathbb{E}[|\bar{q}(C, a)|] \leq \mathbb{E}[|U(H, a)|] < \infty.$$

The same argument gives

$$\mathbb{E}[|q(C, Z, a)|] < \infty.$$

Hence all expectations below are finite.

Definition 2 (Flat and stage-conditioned decision values). Adopt Assumption 1 and Definition 1. We define

$$V_{\text{flat}} := \mathbb{E}\left[\max_{a \in \mathcal{A}} \bar{q}(C, a)\right], \quad V_{\text{stage}} := \mathbb{E}\left[\max_{a \in \mathcal{A}} q(C, Z, a)\right].$$

These are exactly the appendix versions of the quantities appearing in Theorem 1 in the main text. Equivalently,

$$V_{\text{flat}} = \mathbb{E}\left[\max_{a \in \mathcal{A}} \mathbb{E}[U(H, a) \mid C]\right], \quad V_{\text{stage}} = \mathbb{E}\left[\max_{a \in \mathcal{A}} \mathbb{E}[U(H, a) \mid C, Z]\right].$$

The following lemma is the only structural identity needed in the proof of Theorem 1.

Lemma 1 (Collapsing stage-conditioned utility to the flat information structure). Adopt Assumption 1 and Definition 1. Then for every action $a \in \mathcal{A}$,

$$\bar{q}(C, a) = \sum_{z=1}^K p(z \mid C) q(C, z, a) \quad \text{almost surely.}$$

Proof. Fix an arbitrary action $a \in \mathcal{A}$. Define the random variable

$$R_a := \sum_{z=1}^K p(z \mid C) q(C, z, a).$$

We will prove that R_a is a version of $\mathbb{E}[U(H, a) \mid \sigma(C)]$. Since $\bar{q}(C, a)$ is also a version of $\mathbb{E}[U(H, a) \mid \sigma(C)]$ by Definition 1, the conclusion will follow.

We first note that R_a is $\sigma(C)$ -measurable. Indeed, for each fixed z , both $p(z \mid C)$ and $q(C, z, a)$ are $\sigma(C)$ -measurable, so their product is $\sigma(C)$ -measurable, and a finite sum of such terms is again $\sigma(C)$ -measurable.

It remains to verify the defining property of conditional expectation. Let G be any bounded $\sigma(C)$ -measurable random variable. We must show that

$$\mathbb{E}[GR_a] = \mathbb{E}[GU(H, a)].$$

Starting from the left-hand side and expanding the definition of R_a , we obtain

$$\begin{aligned} \mathbb{E}[GR_a] &= \mathbb{E}\left[G \sum_{z=1}^K p(z \mid C) q(C, z, a)\right] \\ &= \sum_{z=1}^K \mathbb{E}[G p(z \mid C) q(C, z, a)]. \end{aligned}$$

For each fixed z , the random variable $Gq(C, z, a)$ is $\sigma(C)$ -measurable and integrable. Therefore, by the defining property of conditional expectation,

$$\begin{aligned} \mathbb{E}[G p(z \mid C) q(C, z, a)] &= \mathbb{E}[G q(C, z, a) \mathbb{E}[\mathbf{1}\{Z = z\} \mid \sigma(C)]] \\ &= \mathbb{E}[G q(C, z, a) \mathbf{1}\{Z = z\}]. \end{aligned}$$

Summing over z yields

$$\begin{aligned} \mathbb{E}[GR_a] &= \sum_{z=1}^K \mathbb{E}[G q(C, z, a) \mathbf{1}\{Z = z\}] \\ &= \mathbb{E}\left[G \sum_{z=1}^K \mathbf{1}\{Z = z\} q(C, z, a)\right] \\ &= \mathbb{E}[Gq(C, Z, a)] \end{aligned}$$

By Definition 1,

$$q(C, Z, a) = \mathbb{E}[U(H, a) \mid \sigma(C, Z)] \quad \text{almost surely.}$$

Because G is $\sigma(C)$ -measurable and $\sigma(C) \subseteq \sigma(C, Z)$, the random variable G is also $\sigma(C, Z)$ -measurable. Applying the defining property of conditional expectation once more gives

$$\mathbb{E}[Gq(C, Z, a)] = \mathbb{E}[GU(H, a)].$$

Combining the previous displays, we conclude that

$$\mathbb{E}[GR_a] = \mathbb{E}[GU(H, a)]$$

for every bounded $\sigma(C)$ -measurable random variable G . This finishes the proof. $\square$

$\square$

Definition 3 (Aliasing gap). *Adopt Assumption 1 and Definition 1. For $c \in C$, define the stage-aliasing gap*

$$\Delta_{\text{alias}}(c) := \sum_{z=1}^K p(z | c) \max_{a \in \mathcal{A}} q(c, z, a) - \max_{a \in \mathcal{A}} \sum_{z=1}^K p(z | c) q(c, z, a).$$

Since $\mathcal{A}$ and $[K]$ are finite and the chosen versions p, q are measurable, the function $\Delta_{\text{alias}} : C \rightarrow \mathbb{R}$ is measurable.

We can now restate and strengthen the theorem from the main text.

Theorem 2 (Restatement and strengthening of Theorem 1). *Adopt Assumption 1 and Definitions 1–3. Then*

$$V_{\text{stage}} - V_{\text{flat}} = \mathbb{E}[\Delta_{\text{alias}}(C)] \geq 0.$$

Consequently,

$$V_{\text{stage}} \geq V_{\text{flat}}.$$

Moreover, for $\mathbb{P}_C$ -almost every $c \in C$, the following are equivalent:

1. $\Delta_{\text{alias}}(c) = 0$.
2. *There exists an action*

$$a_c^* \in \bigcap_{z \in S(c)} \arg \max_{a \in \mathcal{A}} q(c, z, a).$$

In particular, if there exists a measurable set $C_0 \subseteq C$ with

$$\mathbb{P}_C(C_0) > 0$$

and two distinct stages $z \neq z'$ such that for every $c \in C_0$,

$$p(z | c) > 0, \quad p(z' | c) > 0, \quad \arg \max_{a \in \mathcal{A}} q(c, z, a) \cap \arg \max_{a \in \mathcal{A}} q(c, z', a) = \emptyset,$$

then

$$V_{\text{stage}} > V_{\text{flat}}.$$

Proof. First, we rewrite V_{stage} and V_{flat} to show their subtraction form as in $\mathbb{E}[\Delta_{\text{alias}}(C)]$. By Definition 2,

$$V_{\text{stage}} = \mathbb{E} \left[\max_{a \in \mathcal{A}} q(C, Z, a) \right].$$

For each fixed $z \in [K]$, define

$$M_z(C) := \max_{a \in \mathcal{A}} q(C, z, a).$$

Since $\mathcal{A}$ is finite and $q(\cdot, z, a)$ is measurable for each a , the random variable $M_z(C)$ is $\sigma(C)$ -measurable. Also, because Z takes values in the finite set $[K]$,

$$\max_{a \in \mathcal{A}} q(C, Z, a) = \sum_{z=1}^K \mathbf{1}\{Z = z\} M_z(C) \quad \text{almost surely.}$$

Taking conditional expectation given $\sigma(C)$, we obtain

$$\begin{aligned} \mathbb{E} \left[\max_{a \in \mathcal{A}} q(C, Z, a) \mid \sigma(C) \right] &= \mathbb{E} \left[\sum_{z=1}^K \mathbf{1}\{Z = z\} M_z(C) \mid \sigma(C) \right] \\ &= \sum_{z=1}^K \mathbb{E} [\mathbf{1}\{Z = z\} M_z(C) \mid \sigma(C)] \\ &= \sum_{z=1}^K M_z(C) \mathbb{E} [\mathbf{1}\{Z = z\} \mid \sigma(C)] \\ &= \sum_{z=1}^K p(z \mid C) M_z(C). \end{aligned}$$

Taking expectation once more and using the tower property gives

$$V_{\text{stage}} = \mathbb{E} \left[\sum_{z=1}^K p(z \mid C) \max_{a \in \mathcal{A}} q(C, z, a) \right].$$

We next rewrite V_{flat} . By Definition 2,

$$V_{\text{flat}} = \mathbb{E} \left[\max_{a \in \mathcal{A}} \bar{q}(C, a) \right].$$

By Lemma 1,

$$\bar{q}(C, a) = \sum_{z=1}^K p(z \mid C) q(C, z, a) \quad \text{almost surely.}$$

Substituting this identity into the preceding display yields

$$V_{\text{flat}} = \mathbb{E} \left[\max_{a \in \mathcal{A}} \sum_{z=1}^K p(z \mid C) q(C, z, a) \right].$$

Then we can simply see that by Definition 3

$$V_{\text{stage}} - V_{\text{flat}} = \mathbb{E} \left[\sum_{z=1}^K p(z \mid C) \max_{a \in \mathcal{A}} q(C, z, a) - \max_{a \in \mathcal{A}} \sum_{z=1}^K p(z \mid C) q(C, z, a) \right] = \mathbb{E} [\Delta_{\text{alias}}(C)].$$

It remains to show that $\Delta_{\text{alias}}(c) \geq 0$ for every c . Fix an arbitrary $c \in C$. For every action $a \in \mathcal{A}$ and every stage $z \in [K]$, we have

$$q(c, z, a) \leq \max_{a' \in \mathcal{A}} q(c, z, a').$$

Multiplying both sides by the nonnegative quantity $p(z \mid c)$ and summing over z gives

$$\sum_{z=1}^K p(z \mid c) q(c, z, a) \leq \sum_{z=1}^K p(z \mid c) \max_{a' \in \mathcal{A}} q(c, z, a').$$

Since this inequality holds for every $a \in \mathcal{A}$, it continues to hold after taking the maximum over a on the left-hand side, and this means that $\Delta_{\text{alias}}(c) \geq 0$. Therefore

$$V_{\text{stage}} - V_{\text{flat}} = \mathbb{E} [\Delta_{\text{alias}}(C)] \geq 0,$$

which implies

$$V_{\text{stage}} \geq V_{\text{flat}}.$$

Then we are ready to make characterizations when the stage-aliasing gap is zero. Since $\mathcal{A}$ is finite and nonempty, there exists at least one action

$$a_c^{\text{flat}} \in \arg \max_{a \in \mathcal{A}} \sum_{z=1}^K p(z | c) q(c, z, a).$$

By Definition 3,

$$\begin{aligned} \Delta_{\text{alias}}(c) &= \sum_{z=1}^K p(z | c) M_z(c) - \sum_{z=1}^K p(z | c) q(c, z, a_c^{\text{flat}}) \\ &= \sum_{z=1}^K p(z | c) (M_z(c) - q(c, z, a_c^{\text{flat}})). \end{aligned}$$

Each summand on the right-hand side is nonnegative, because $M_z(c)$ is the maximum of $q(c, z, \cdot)$ over $\mathcal{A}$. We now prove the two directions separately.

(i) If $\Delta_{\text{alias}}(c) = 0$, then there exists a common maximizer over $S(c)$. This implies that

$$\sum_{z=1}^K p(z | c) (M_z(c) - q(c, z, a_c^{\text{flat}})) = 0.$$

Every term in the sum is nonnegative. Therefore, for every z such that $p(z | c) > 0$, we must have

$$M_z(c) - q(c, z, a_c^{\text{flat}}) = 0,$$

i.e.,

$$q(c, z, a_c^{\text{flat}}) = M_z(c) = \max_{a \in \mathcal{A}} q(c, z, a).$$

Hence

$$a_c^{\text{flat}} \in \arg \max_{a \in \mathcal{A}} q(c, z, a) \quad \text{for every } z \in S(c).$$

Equivalently,

$$a_c^{\text{flat}} \in \bigcap_{z \in S(c)} \arg \max_{a \in \mathcal{A}} q(c, z, a),$$

so a common maximizer exists.

(ii) If there exists a common maximizer over $S(c)$, then $\Delta_{\text{alias}}(c) = 0$.

Conversely, assume there exists an action

$$a_c^{\star} \in \bigcap_{z \in S(c)} \arg \max_{a \in \mathcal{A}} q(c, z, a).$$

Then for every $z \in S(c)$,

$$q(c, z, a_c^{\star}) = M_z(c).$$

For $z \notin S(c)$, we have $p(z | c) = 0$, so those stages contribute nothing to any weighted sum below. Hence

$$\sum_{z=1}^K p(z | c) q(c, z, a_c^{\star}) = \sum_{z=1}^K p(z | c) M_z(c).$$

Since a_c^* is one feasible action in the maximization,

$$\max_{a \in \mathcal{A}} \sum_{z=1}^K p(z | c) q(c, z, a) \geq \sum_{z=1}^K p(z | c) q(c, z, a_c^*) = \sum_{z=1}^K p(z | c) M_z(c).$$

On the other hand, the following is obvious and has been shown above

$$\max_{a \in \mathcal{A}} \sum_{z=1}^K p(z | c) q(c, z, a) \leq \sum_{z=1}^K p(z | c) M_z(c).$$

Therefore equality holds:

$$\max_{a \in \mathcal{A}} \sum_{z=1}^K p(z | c) q(c, z, a) = \sum_{z=1}^K p(z | c) M_z(c),$$

which is exactly

$$\Delta_{\text{alias}}(c) = 0.$$

Combining (i) and (ii), we conclude that for every $c \in C$,

$$\Delta_{\text{alias}}(c) = 0 \iff \bigcap_{z \in S(c)} \arg \max_{a \in \mathcal{A}} q(c, z, a) \neq \emptyset.$$

Since all objects are defined only up to $\mathbb{P}_C$ -null sets through the chosen versions of conditional expectations and conditional probabilities, the equivalence is interpreted for $\mathbb{P}_C$ -almost every c .

Finally, we can prove the strict inequality criterion. Assume there exists a measurable set $C_0 \subseteq C$ such that

$$\mathbb{P}_C(C_0) > 0$$

and two distinct stages $z \neq z'$ such that for every $c \in C_0$,

$$p(z | c) > 0, \quad p(z' | c) > 0, \quad \arg \max_{a \in \mathcal{A}} q(c, z, a) \cap \arg \max_{a \in \mathcal{A}} q(c, z', a) = \emptyset.$$

By the zero-gap equivalence we proved above, and that we know $\Delta_{\text{alias}}(c) \geq 0$, this means that $\Delta_{\text{alias}}(C) \geq 0$ almost surely, and thus

$$V_{\text{stage}} > V_{\text{flat}}.$$

This completes the proof. □

Remark 2 (Interpretation). *Theorem 2 strengthens the main-text theorem in two ways. First, it identifies the exact gain from making the stage label explicit:*

$$V_{\text{stage}} - V_{\text{flat}} = \mathbb{E}[\Delta_{\text{alias}}(C)].$$

Second, it shows that the gain is strict exactly when the compressed context C aliases decision points whose stage-conditioned optimal actions disagree, which is the usual case. This is the precise sense in which explicit stage structure helps reasoning under compressed local context.

E.2. Judge-Aligned Stage-Weighted Credit Assignment

This subsection formalizes the main-text intuition behind Theorem 3. As in Appendix E.1, we work on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. However, unlike Appendix E.1, which studies a single random decision point, we now analyze full rollouts and their policy-gradient signals.

Setup and notation. We suppress the rollout index i used in Section 3.3 and analyze a single generic rollout. Let τ denote a rollout sampled from a policy-induced distribution $p_\theta(\tau)$, and let $\mathcal{B}_k$ denote the token set of stage k , matching the notation in the main text. For each token step t , let

$$H_t := (q, a_{<t}, o_{<t})$$

denote the random history before taking action a_t , and define the stage score-function sum

$$\Gamma_k := \sum_{t \in \mathcal{B}_k} \nabla_\theta \log \pi_\theta(a_t | H_t).$$

We assume $\pi_\theta(a | h)$ is differentiable in θ , the environment dynamics are independent of θ , and Γ_k is square-integrable for every k .

For each stage $k \in [K]$, let

$$R_k : \Omega \rightarrow \mathbb{R}$$

denote the observed stagewise judge score, and let

$$Y_k : \Omega \rightarrow \mathbb{R}$$

denote a latent true process score for stage k . Both are assumed integrable.

As in Section 3.3, let

$$\Lambda = (\lambda_{k,j}) \in [0, 1]^{K \times K}$$

be a causal stage-weight matrix satisfying

$$\lambda_{k,j} = 0 \quad \text{for } j < k, \quad \lambda_{k,k} = 1.$$

Definitions. For each stage k , define the oracle process-level gradient contribution

$$g_k^\star := \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k Y_j]. \tag{2}$$

This is the stage- k gradient contribution that would be induced by the stage-dependent return $G_{i,k}^\Lambda := \sum_{j=k}^K \lambda_{k,j} R_{i,j}$ if the latent true stage scores were directly observable.

Define the judge-induced stage-weighted signal

$$g_k^\Lambda := \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k R_j], \tag{3}$$

and the terminal-broadcast signal

$$g_k^{\text{term}} := \mathbb{E}[\Gamma_k R_K], \tag{4}$$

Finally, define

$$M_k^\Lambda := \left\| \mathbb{E}[\Gamma_k Y_K] - \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k Y_j] \right\|_2. \quad (5)$$

The quantity M_k^Λ measures how much oracle process-level signal is omitted when one uses only the final-stage score Y_K instead of the full stage-weighted oracle target.

Assumption 2 (Judge alignment). *For each stage pair $k \leq j$, there exists a constant $\epsilon_{k,j} \geq 0$ such that*

$$\left\| \mathbb{E}[\Gamma_k (R_j - Y_j)] \right\|_2 \leq \epsilon_{k,j}. \quad (6)$$

That is, the observed judge score R_j approximates the latent true score Y_j in the gradient-relevant direction defined by Γ_k .

Theorem 3 (Benefit of stage-weighted credit, informal). *If the omitted true intermediate signal outweighs the cumulative judge misalignment, then stage-weighted credit yields a strictly better gradient approximation:*

$$\|g_k^\Lambda - g_k^\star\|_2 < \|g_k^{\text{term}} - g_k^\star\|_2.$$

Theorem 4 (Formal version of Theorem 3). *Under Assumption 2, the following hold for every stage $k \in [K]$.*

1. *Error of the stage-weighted signal relative to the oracle target:*

$$\|g_k^\Lambda - g_k^\star\|_2 \leq \sum_{j=k}^K \lambda_{k,j} \epsilon_{k,j}.$$

2. *Lower bound for terminal broadcast relative to the oracle target:*

$$\|g_k^{\text{term}} - g_k^\star\|_2 \geq M_k^\Lambda - \epsilon_{k,K}.$$

3. *Comparison. If*

$$\sum_{j=k}^K \lambda_{k,j} \epsilon_{k,j} < M_k^\Lambda - \epsilon_{k,K},$$

then

$$\|g_k^\Lambda - g_k^\star\|_2 < \|g_k^{\text{term}} - g_k^\star\|_2.$$

In particular, when judge misalignment is sufficiently small relative to the omitted intermediate-stage oracle signal, the stage-weighted signal is strictly closer than terminal broadcast to the intended process-level gradient contribution.

Proof. Fix a stage k . **First, we show (1).** By definition 3 and 2,

$$g_k^\Lambda - g_k^\star = \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k R_j] - \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k Y_j] = \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k (R_j - Y_j)].$$

Taking the Euclidean norm and applying the triangle inequality yields

$$\|g_k^\Lambda - g_k^\star\|_2 \leq \sum_{j=k}^K \lambda_{k,j} \left\| \mathbb{E}[\Gamma_k (R_j - Y_j)] \right\|_2.$$

Applying Assumption 2 results in the desired form,

$$\|g_k^\Lambda - g_k^\star\|_2 \leq \sum_{j=k}^K \lambda_{k,j} \epsilon_{k,j}.$$

Second, we try to prove (2). By definition 4 and 2,

$$g_k^{\text{term}} - g_k^\star = \mathbb{E}[\Gamma_k R_K] - \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k Y_j] = \underbrace{\mathbb{E}[\Gamma_k (R_K - Y_K)]}_{=: E_k} + \underbrace{\left(\mathbb{E}[\Gamma_k Y_K] - \sum_{j=k}^K \lambda_{k,j} \mathbb{E}[\Gamma_k Y_j] \right)}_{=: D_k}.$$

From definition 5 we know that $\|D_k\|_2 = M_k^\Lambda$. Applying the reverse triangle inequality,

$$\|E_k + D_k\|_2 \geq \|D_k\|_2 - \|E_k\|_2,$$

we obtain

$$\|g_k^{\text{term}} - g_k^\star\|_2 \geq M_k^\Lambda - \|\mathbb{E}[\Gamma_k (R_K - Y_K)]\|_2.$$

Using Assumption 2 with $j = K$,

$$\|\mathbb{E}[\Gamma_k (R_K - Y_K)]\|_2 \leq \epsilon_{k,K}.$$

Hence

$$\|g_k^{\text{term}} - g_k^\star\|_2 \geq M_k^\Lambda - \epsilon_{k,K}.$$

Finally, we are ready to state the last result. We have shown that

$$\|g_k^\Lambda - g_k^\star\|_2 \leq \sum_{j=k}^K \lambda_{k,j} \epsilon_{k,j}, \quad \|g_k^{\text{term}} - g_k^\star\|_2 \geq M_k^\Lambda - \epsilon_{k,K}.$$

Therefore, if

$$\sum_{j=k}^K \lambda_{k,j} \epsilon_{k,j} < M_k^\Lambda - \epsilon_{k,K},$$

then

$$\|g_k^\Lambda - g_k^\star\|_2 < \|g_k^{\text{term}} - g_k^\star\|_2.$$

This proves item 3 and completes the proof. $\square$

Corollary 1 (A sufficient condition from score MSE). *Suppose in addition that for some constants $B_k \geq 0$ and $\delta_j \geq 0$,*

$$\mathbb{E}[\|\Gamma_k\|_2^2] \leq B_k, \quad \mathbb{E}[(R_j - Y_j)^2] \leq \delta_j^2.$$

Then Assumption 2 holds with

$$\epsilon_{k,j} := \sqrt{B_k} \delta_j.$$

Consequently,

$$\|g_k^\Lambda - g_k^\star\|_2 \leq \sqrt{B_k} \sum_{j=k}^K \lambda_{k,j} \delta_j,$$

and

$$\|g_k^{\text{term}} - g_k^\star\|_2 \geq M_k^\Lambda - \sqrt{B_k} \delta_K.$$

Proof. Fix $k \leq j$. By Cauchy–Schwarz,

$$\begin{aligned} \|\mathbb{E}[\Gamma_k(R_j - Y_j)]\|_2 &\leq \mathbb{E}[\|\Gamma_k\|_2 |R_j - Y_j|] \\ &\leq \left(\mathbb{E}[\|\Gamma_k\|_2^2]\right)^{1/2} \left(\mathbb{E}[(R_j - Y_j)^2]\right)^{1/2} \\ &\leq \sqrt{B_k} \delta_j. \end{aligned}$$

Thus Assumption 2 holds with $\epsilon_{k,j} = \sqrt{B_k} \delta_j$. The displayed bounds then follow immediately from Theorem 4. $\square$

Remark 3. Theorem 4 and Corollary 1 highlight a fundamental trade-off: the advantage of stage-weighted credit depends on the strength of the omitted intermediate-stage oracle signal M_k^Λ versus the quality of judge alignment, captured by the score errors δ_j . Crucially, this establishes that as long as the intermediate stages contain sufficient true process signal (i.e., M_k^Λ is sufficiently large), the benefit of capturing this dense signal strictly outweighs the accumulated noise from an imperfect intermediate judge $\delta_j > 0$. In such regimes, stage-weighted credit serves as a strictly better approximation of the intended process objective than relying solely on terminal broadcast.

E.3. Judge-Gated Co-Evolution of Policy and Rubric Bank

This subsection formalizes the third component of SCRIBE in Section 3.4. Unlike Appendix E.1, which studies the value of explicit stage information, and Appendix E.2, which studies stagewise credit assignment, the object here is the self-evolution loop itself: the same shared backbone both solves the current task and produces rubric-grounded reflections that can later be reused within-episode and across episodes.

The formalization below matches our implementation. The task objective updates trajectory tokens, whereas the reflection objective is computed on reflection tokens only. Concretely, the selected rollout is treated as a fixed conditioning context during the reflection update, so the reflection-utility gradient does not backpropagate through the sampled trajectory. The co-evolution effect therefore comes from parameter sharing: task updates and reflection updates act on the same backbone parameters, even though they are computed from different generated token blocks.

The key assumption below is a *judge-gated local positive-transfer* condition. This is not intended as a global claim that every generated reflection is always useful throughout training. Rather, it isolates the local regime in which reflections that are *accepted by the judge* behave like a helpful auxiliary objective. Similar gradient-similarity and positive-transfer conditions are standard in the auxiliary-learning and multi-task optimization literature: helpful auxiliary objectives tend to align with the main-task gradient, whereas misaligned gradients induce negative transfer (Du et al., 2019; Liu et al., 2021; Wang et al., 2021; Wu et al., 2020; Yu et al., 2020). At the same time, not all task combinations are beneficial to train together (Standley et al., 2020). This perspective is also consistent with theory on shared representation learning and task relevance in transfer (Chen et al., 2022; Maurer et al., 2016; Tripuraneni et al., 2020).

Assumption 3 (Setup for judge-gated co-evolution). *Throughout this subsection, let $\Theta \subseteq \mathbb{R}^d$ be an open parameter domain. The following assumptions hold.*

1. **Query distribution for the task objective.** *There is a query random variable*

$$Q \sim \mathcal{D}.$$

2. **Current rubric bank.** At the current training step, the rubric bank is a fixed measurable object $\mathcal{M}$. This corresponds to analyzing a single parameter update while treating the bank state as fixed.
3. **Task rollout distribution.** Given query $Q = q$ and bank $\mathcal{M}$, a rollout

$$T \sim p_\theta(\cdot \mid q, \mathcal{M})$$

is sampled from the deployed task policy.

4. **Fixed reflection-context distribution.** There exists a fixed measurable distribution ξ over query-trajectory pairs

$$(\tilde{Q}, \tilde{T}) \sim \xi.$$

One may think of ξ as the distribution of trajectories selected for reflection training at the current update, e.g., after first sampling rollout groups under the current behavior policy and then selecting one trajectory per query. For the local analysis in this subsection, ξ is treated as fixed. This matches the implementation where the reflection objective applies gradients only on reflection tokens and treats the selected trajectory as a fixed prompt / context.

5. **Shared-backbone reflection generation.** Given $(\tilde{Q}, \tilde{T}) = (q, \tau)$ and bank $\mathcal{M}$, a rubric-grounded reflection

$$S \sim r_\theta(\cdot \mid q, \tau, \mathcal{M})$$

is sampled. Although we write p_θ and r_θ separately for clarity, both are induced by the same underlying autoregressive backbone π_θ under different prompts / contexts.

6. **Task score, reflection-utility scores, and judge gate.** There exist measurable, integrable random quantities

$$R(Q, T; \mathcal{M}) \in \mathbb{R}, \quad \Delta^w(\tilde{Q}, \tilde{T}, S; \mathcal{M}) \in \mathbb{R}, \quad \Delta^c(\tilde{Q}, \tilde{T}, S; \mathcal{M}) \in \mathbb{R},$$

and a measurable acceptance indicator

$$A(\tilde{Q}, \tilde{T}, S; \mathcal{M}) \in \{0, 1\}.$$

Here R is the judged task score of the deployed rollout, Δ^w is the judged within-episode usefulness of the reflection, Δ^c is the judged cross-episode transfer usefulness, and $A = 1$ means that the reflection is judged sufficiently useful to be accepted for downstream adaptation / storage. These objects do not depend directly on θ except through the sampled random variables and the fixed bank $\mathcal{M}$.

7. **Differentiability and score-function regularity.** For $\mathcal{D}$ -almost every q , the conditional distribution $p_\theta(\tau \mid q, \mathcal{M})$ is differentiable in θ . For ξ -almost every (q, τ) , the conditional distribution $r_\theta(s \mid q, \tau, \mathcal{M})$ is differentiable in θ . Differentiation may be interchanged with the expectations below, and all score-function terms introduced later are square-integrable.
8. **Smoothness.** The objectives J_{task} and U defined below are continuously differentiable. Moreover, J_{task} is L_J -smooth and U is L_U -smooth on Θ .

Definition 4 (Task objective and judge-gated memory objective). Adopt Assumption 3. Define the task-rollout and reflection score-function sums

$$\Gamma^{\text{traj}} := \nabla_\theta \log p_\theta(T \mid Q, \mathcal{M}), \quad \Gamma^{\text{ref}} := \nabla_\theta \log r_\theta(S \mid \tilde{Q}, \tilde{T}, \mathcal{M}).$$

Because both conditional distributions are induced by the same backbone π_θ , these admit tokenwise decompositions

$$\Gamma^{\text{traj}} = \sum_{t=1}^{|T|} \nabla_\theta \log \pi_\theta(a_t \mid h_t), \quad \Gamma^{\text{ref}} = \sum_{u=1}^{|S|} \nabla_\theta \log \pi_\theta(s_u \mid c_u),$$

for the appropriate rollout histories h_t and reflection-generation contexts c_u .

For weights $\beta_w, \beta_c \geq 0$, define

$$J_{\text{task}}(\theta) := \mathbb{E}[R(Q, T; \mathcal{M})],$$

where the expectation is with respect to

$$Q \sim \mathcal{D}, \quad T \sim p_\theta(\cdot \mid Q, \mathcal{M}),$$

and define

$$U(\theta) := \mathbb{E}\left[A(\tilde{Q}, \tilde{T}, S; \mathcal{M})\left(\beta_w \Delta^w(\tilde{Q}, \tilde{T}, S; \mathcal{M}) + \beta_c \Delta^c(\tilde{Q}, \tilde{T}, S; \mathcal{M})\right)\right],$$

where the expectation is with respect to

$$(\tilde{Q}, \tilde{T}) \sim \xi, \quad S \sim r_\theta(\cdot \mid \tilde{Q}, \tilde{T}, \mathcal{M}).$$

We also define the combined co-evolution objective

$$J_{\text{coevo}}(\theta) := J_{\text{task}}(\theta) + U(\theta).$$

Assumption 4 (Judge-gated local positive transfer). Adopt Assumption 3 and Definition 4. Let

$$g := \nabla J_{\text{task}}(\theta),$$

and define the ungated reflection-utility score-gradient random vector

$$\Psi := \Gamma^{\text{ref}}\left(\beta_w \Delta^w(\tilde{Q}, \tilde{T}, S; \mathcal{M}) + \beta_c \Delta^c(\tilde{Q}, \tilde{T}, S; \mathcal{M})\right).$$

Assume there exist constants $p_0 \in (0, 1]$ and $\mu > 0$ such that

$$\mathbb{P}(A = 1) \geq p_0, \quad \langle g, \mathbb{E}[\Psi \mid A = 1] \rangle \geq \mu.$$

That is, reflections that pass the judge are accepted with nontrivial probability, and their conditional expected gradient contribution is positively aligned with the task gradient.

Theorem 5 (Judge-gated shared-backbone co-evolution). *Adopt Assumptions 3–4 and Definition 4. Let*

$$g := \nabla J_{\text{task}}(\theta), \quad h := \nabla U(\theta).$$

Then the following hold.

1. **Mutual improvement.** *For every step size $\eta > 0$,*

$$U(\theta + \eta g) - U(\theta) \geq \eta p_0 \mu - \frac{L_U \eta^2}{2} \|g\|_2^2,$$

and

$$J_{\text{task}}(\theta + \eta h) - J_{\text{task}}(\theta) \geq \eta p_0 \mu - \frac{L_J \eta^2}{2} \|h\|_2^2.$$

In particular, if

$$0 < \eta < \min \left\{ \frac{2p_0\mu}{L_U \|g\|_2^2}, \frac{2p_0\mu}{L_J \|h\|_2^2} \right\},$$

then a task-improving step also improves the judge-gated memory objective, and a memory-improving step also improves the task objective.

2. **Dominance over task-only training with static memory.** *Consider the task-only update*

$$\theta_{\text{stat}}^+ := \theta + \eta g,$$

which updates the deployed task policy while assigning zero explicit training signal to reflection quality, and the co-evolution update

$$\theta_{\text{co}}^+ := \theta + \eta(g + h).$$

Then

$$J_{\text{task}}(\theta_{\text{co}}^+) - J_{\text{task}}(\theta_{\text{stat}}^+) \geq \eta p_0 \mu - L_J \eta^2 \|g\|_2 \|h\|_2 - \frac{L_J \eta^2}{2} \|h\|_2^2.$$

Consequently, if the reflection is sufficiently good,

$$p_0 \mu > L_J \eta \left(\|g\|_2 \|h\|_2 + \frac{1}{2} \|h\|_2^2 \right),$$

then

$$J_{\text{task}}(\theta_{\text{co}}^+) > J_{\text{task}}(\theta_{\text{stat}}^+).$$

Proof. To prove the desired statement, we first derive the formula for g and h . Let $\mu_\theta(dq, d\tau) := \mathcal{D}(dq) p_\theta(d\tau | q, \mathcal{M})$ denote the joint law of (Q, T) . By Definition 4,

$$J_{\text{task}}(\theta) = \int R(q, \tau; \mathcal{M}) \mu_\theta(dq, d\tau).$$

By Assumption 3, differentiation may be interchanged with the integral. Since $\mathcal{D}$ does not depend on

θ , by the usual log-derivative trick,

$$\begin{aligned}\nabla J_{\text{task}}(\theta) &= \int R(q, \tau; \mathcal{M}) \nabla p_{\theta}(\tau | q, \mathcal{M}) \mathcal{D}(dq) d\tau \\ &= \int R(q, \tau; \mathcal{M}) p_{\theta}(\tau | q, \mathcal{M}) \nabla \log p_{\theta}(\tau | q, \mathcal{M}) \mathcal{D}(dq) d\tau \\ &= \mathbb{E}[\Gamma^{\text{traj}} R(Q, T; \mathcal{M})].\end{aligned}$$

We next derive the formula for h . Write

$$\Delta(q, \tau, s; \mathcal{M}) := \beta_w \Delta^w(q, \tau, s; \mathcal{M}) + \beta_c \Delta^c(q, \tau, s; \mathcal{M}),$$

so that

$$U(\theta) = \mathbb{E}[A(\tilde{Q}, \tilde{T}, S; \mathcal{M}) \Delta(\tilde{Q}, \tilde{T}, S; \mathcal{M})].$$

Let

$$\nu_{\theta}(dq, d\tau, ds) := \xi(dq, d\tau) r_{\theta}(ds | q, \tau, \mathcal{M})$$

denote the joint law of $(\tilde{Q}, \tilde{T}, S)$. Then

$$U(\theta) = \int A(q, \tau, s; \mathcal{M}) \Delta(q, \tau, s; \mathcal{M}) \nu_{\theta}(dq, d\tau, ds).$$

Differentiating under the integral sign gives

$$\begin{aligned}\nabla U(\theta) &= \int A(q, \tau, s; \mathcal{M}) \Delta(q, \tau, s; \mathcal{M}) \xi(dq, d\tau) \nabla r_{\theta}(s | q, \tau, \mathcal{M}) ds \\ &= \int A(q, \tau, s; \mathcal{M}) \Delta(q, \tau, s; \mathcal{M}) \xi(dq, d\tau) r_{\theta}(s | q, \tau, \mathcal{M}) \nabla \log r_{\theta}(s | q, \tau, \mathcal{M}) ds \\ &= \mathbb{E}[A(\tilde{Q}, \tilde{T}, S; \mathcal{M}) \Gamma^{\text{ref}} \Delta(\tilde{Q}, \tilde{T}, S; \mathcal{M})] \\ &= \mathbb{E}[A\Psi] := h.\end{aligned}$$

By the definition of A and Assumption 4,

$$\mathbb{E}[A\Psi] = \mathbb{P}(A = 1) \mathbb{E}[\Psi | A = 1].$$

and that

$$\langle g, h \rangle = \langle g, \mathbb{P}(A = 1) \mathbb{E}[\Psi | A = 1] \rangle \quad (7)$$

$$= \mathbb{P}(A = 1) \langle g, \mathbb{E}[\Psi | A = 1] \rangle \quad (8)$$

$$\geq p_0 \mu, \quad (9)$$

By the assumption 3, U is L_U -smooth, for every vector $v \in \mathbb{R}^d$ we have the standard lower bound

$$U(\theta + v) \geq U(\theta) + \langle \nabla U(\theta), v \rangle - \frac{L_U}{2} \|v\|_2^2.$$

Applying this with $v = \eta g$ and using Eq. 9 yields

$$U(\theta + \eta g) - U(\theta) \geq \eta \langle h, g \rangle - \frac{L_U \eta^2}{2} \|g\|_2^2 \geq \eta p_0 \mu - \frac{L_U \eta^2}{2} \|g\|_2^2.$$

Likewise, because J_{task} is L_J -smooth,

$$J_{\text{task}}(\theta + \eta h) - J_{\text{task}}(\theta) \geq \eta \langle g, h \rangle - \frac{L_J \eta^2}{2} \|h\|_2^2 \geq \eta p_0 \mu - \frac{L_J \eta^2}{2} \|h\|_2^2.$$

If

$$0 < \eta < \min \left\{ \frac{2p_0\mu}{L_U \|g\|_2^2}, \frac{2p_0\mu}{L_J \|h\|_2^2} \right\},$$

then both right-hand sides are strictly positive.

To prove the other part, define

$$\phi(t) := J_{\text{task}}(\theta_{\text{stat}}^+ + t\eta h), \quad t \in [0, 1].$$

Then

$$\phi(1) - \phi(0) = J_{\text{task}}(\theta_{\text{co}}^+) - J_{\text{task}}(\theta_{\text{stat}}^+) = \int_0^1 \phi'(t) dt.$$

By the chain rule,

$$\phi'(t) = \eta \langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+ + t\eta h), h \rangle.$$

Since ∇J_{task} is L_J -Lipschitz,

$$\|\nabla J_{\text{task}}(\theta_{\text{stat}}^+ + t\eta h) - \nabla J_{\text{task}}(\theta_{\text{stat}}^+)\|_2 \leq L_J t \eta \|h\|_2.$$

Hence, for every $t \in [0, 1]$, by Cauchy-Schwarz

$$\langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+ + t\eta h), h \rangle \geq \langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+), h \rangle - L_J t \eta \|h\|_2^2.$$

Integrating from 0 to 1 gives

$$J_{\text{task}}(\theta_{\text{co}}^+) - J_{\text{task}}(\theta_{\text{stat}}^+) \geq \eta \langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+), h \rangle - \frac{L_J \eta^2}{2} \|h\|_2^2.$$

It remains to lower-bound the inner product on the right. Again by L_J -Lipschitz continuity of the gradient,

$$\|\nabla J_{\text{task}}(\theta_{\text{stat}}^+) - g\|_2 = \|\nabla J_{\text{task}}(\theta + \eta g) - \nabla J_{\text{task}}(\theta)\|_2 \leq L_J \eta \|g\|_2.$$

Therefore, by Cauchy-Schwarz,

$$\begin{aligned} \langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+), h \rangle &= \langle g, h \rangle + \langle \nabla J_{\text{task}}(\theta_{\text{stat}}^+) - g, h \rangle \\ &\geq \langle g, h \rangle - \|\nabla J_{\text{task}}(\theta_{\text{stat}}^+) - g\|_2 \|h\|_2 \\ &\geq \langle g, h \rangle - L_J \eta \|g\|_2 \|h\|_2 \\ &\geq p_0 \mu - L_J \eta \|g\|_2 \|h\|_2, \end{aligned}$$

where the last step uses Eq. 9. Substituting this bound above yields that

$$J_{\text{task}}(\theta_{\text{co}}^+) - J_{\text{task}}(\theta_{\text{stat}}^+) \geq \eta p_0 \mu - L_J \eta^2 \|g\|_2 \|h\|_2 - \frac{L_J \eta^2}{2} \|h\|_2^2.$$

This proves the desired lower bound. If

$$p_0 \mu > L_J \eta \left(\|g\|_2 \|h\|_2 + \frac{1}{2} \|h\|_2^2 \right),$$

then the right-hand side is strictly positive, and therefore

$$J_{\text{task}}(\theta_{\text{co}}^+) > J_{\text{task}}(\theta_{\text{stat}}^+).$$

□

Remark 4 (Interpretation). *Theorem 5 isolates the source of the co-evolution gain. The key quantity is a judge-gated local condition on the reflections that are actually accepted for reuse. Under this condition, policy training improves accepted reflection utility, and accepted reflection training improves task performance. Moreover, the joint update on $J_{\text{task}} + U$ yields a strictly larger first-order gain in adapted task value than task-only training with a static memory objective. This is the precise sense in which jointly training the evolving memory policy is stronger than inference-time-only static memory. The co-evolution effect survives because g and h still live in the same shared parameter space: a task update changes the future reflection generator, and a reflection update changes the future task policy.*

Remark 5 (Why parameter sharing matters). *The first-order co-evolution mechanism above is specific to the shared-backbone design. If rollout generation and reflection generation were parameterized by disjoint blocks $\theta = (\theta_\pi, \theta_{\text{ref}})$ with J_{task} depending only on θ_π and U depending only on θ_{ref} , then*

$$\nabla J_{\text{task}}(\theta) = (g_\pi, 0), \quad \nabla U(\theta) = (0, h_{\text{ref}}),$$

and hence

$$\langle \nabla J_{\text{task}}(\theta), \nabla U(\theta) \rangle = 0.$$

Thus the mutual-improvement phenomenon in Theorem 5 disappears at first order in the fully decoupled case. This formalizes why a unified backbone can support genuine co-evolution, rather than merely wrapping a static memory module around a separately trained policy.

F. Experiment Details

We largely follow the experimental setup of DR Tulu (Shao et al., 2025) for training, evaluation, infrastructure, and baseline selection, adapting where necessary to support our proposed components. Below, we provide the specific details.

F.1. Training Details

F.1.1. Supervised Fine-Tuning

We fine-tune Qwen3-8B (Yang et al., 2025) using the LLaMA-Factory framework (Zheng et al., 2024) with DeepSpeed ZeRO-3. The SFT data generation process is described in Appendix B.2. Key hyperparameters:

Hyperparameter	Value
Base model	Qwen3-8B
Training epochs	5
Learning rate	4×10^{-5}
LR scheduler	Cosine with 10% warmup
Batch size (per device)	1
Gradient accumulation steps	16
Effective batch size	128 (8 GPUs $\times$ 16 accum.)
Max sequence length	16,384 tokens
Precision	BF16
Weight decay	0.0
Tool output masking	Span masking on <tool_output>
DeepSpeed stage	ZeRO-3
GPUs	8 $\times$ NVIDIA H100 80GB

We apply span masking to tool output tokens so the model does not receive gradient on search

results, learning only to generate its own reasoning, tool calls, and answers.

F.1.2. Reinforcement Learning

We train all RL runs using our modified open-instruct codebase with Ray-based distributed training and vLLM inference engines. The RL setup follows DR. Tulu’s codebase with extensions for stagewise credit assignment and meta-policy training operations. Key hyperparameters:

Hyperparameter	Value
Algorithm	GRPO / SS-GRPO
Rollouts per prompt	8
Unique prompts per step	32
Effective batch size	256 (32×8)
Learning rate	5×10^{-7}
LR scheduler	Constant
KL coefficient (β)	0.001
KL estimator	KL3
Clip ratio (η)	PPO-style clipping
Temperature	1.0
Max response length	18,432 tokens
Max prompt length	8,192 tokens
Max total (pack) length	26,624 tokens
Max tool calls per trajectory	10
DeepSpeed stage	ZeRO-3 with CPU offloading
Gradient checkpointing	Enabled
<i>Rubric judge</i>	
Judge model	Gemini Flash
Rubric generation model	Gemini Flash
Rubric buffer cap (per stage)	3, 2, 2, 3
<i>Rubric bank (full RUBRICEM only)</i>	
Bank embedding model	Qwen3-Embedding-0.6B
Retrieval top- k	2
Reflection trajectories sampled	1
Windowed curriculum K	3
Bank save frequency	Every 10 steps

One additional hyperparameter is the stage-dependent matrix Λ mentioned in Section 3.3, which we set to be

$$\Lambda = \begin{pmatrix} 1.0 & 0.4 & 0.6 & 0.8 \\ 0 & 1.0 & 0.4 & 0.8 \\ 0 & 0 & 1.0 & 0.8 \\ 0 & 0 & 0 & 1.0 \end{pmatrix}.$$

The training data is the same DR. Tulu RL dataset (r1-research/dr-tulu-rl-data), which contains ~ 4.9 K diverse deep research queries. All ablation runs (Baseline RL, SS-GRPO, full RUBRICEM) use the same 600-step budget with 2 nodes, starting from the same RUBRICEM-SFT checkpoint. The final RUBRICEM run uses 4 nodes for longer runs.

Reward design. Since the focus of this work is to improve open-ended answer quality through LLM-as-judge feedback, we use only rubric-based judge signals as RL rewards. Concretely, task-policy

rewards come from the evolving stagewise rubrics used by the judge, and reflection-policy rewards come from judge scores over rubric-grounded reflections. We intentionally do not add auxiliary verifiable rewards, such as format rewards, citation rewards, or tool-use heuristics, which have been used in prior deep research RL recipes such as DR Tulu (Shao et al., 2025). This choice isolates the open-ended component of the problem: improvements should come from better semantic planning, research, synthesis, and experience reuse, rather than from optimizing easily checkable surface constraints. We believe future work could further improve performance by combining rubric-based rewards with calibrated citation, grounding, or formatting rewards, but our current design provides a cleaner and more holistic setting for studying RL beyond verifiable rewards.

F.2. Evaluation Details

F.2.1. Long-Form Benchmarks

We evaluate on four long-form benchmarks:

- **HealthBench** (Arora et al., 2025): Subsampled 1000 medical questions spanning patient consultations, clinical guidelines, and health advice. Evaluation uses an LLM-as-judge (GPT-4) that grades each response against per-question rubrics on multiple axes (accuracy, completeness, context awareness, communication). The reported score is the overall rubric satisfaction rate.
- **ResearchQA** (Yifei et al., 2025): 756 scientific research questions with expert-authored rubric items. Evaluation uses an LLM-as-judge (GPT-4) that assesses coverage of each rubric item on a 5-point scale (Not at all / Barely / Moderately / Mostly / Completely). The reported score is the average normalized coverage.
- **DeepResearchBench (DRB)** (Du et al., 2025): 100 complex research questions requiring long-form reports with citations. Evaluation uses RACE (Report Assessment via Citation Evaluation) scoring, which combines content quality and citation accuracy. The judge (Gemini) evaluates both the substance of the report and whether citations are accurate and well-grounded.
- **ResearchRubrics** (Sharma et al., 2026): 101 open-ended deep research prompts across diverse real-world domains, each paired with expert-written, prompt-specific rubrics. Evaluation uses an LLM-as-judge to assess rubric compliance under fine-grained criteria covering factual grounding, reasoning soundness, synthesis quality, relevance, clarity, and citation use. Rubric items include both positive and negative criteria with different weights, and the reported score is the binary score.

F.2.2. Short-Form Benchmarks

We additionally evaluate on four short-form benchmarks to test out-of-domain transfer:

- **SimpleQA** (Wei et al., 2024): 1,000 factual questions with exact-match evaluation via LLM grading (correct / incorrect / not attempted).
- **2WikiMultihopQA** (Ho et al., 2020): 1000 multi-hop reasoning questions requiring evidence from multiple sources. LLM-as-judge evaluation.
- **WebWalker** (Wu et al., 2025b): 680 web navigation questions. LLM-as-judge evaluation.
- **DeepSearchQA (DSQA)** (Gupta et al., 2026): 900 search-intensive questions with exact-match grading.

All short-form benchmarks are evaluated in zero-shot using the same agent pipeline and search tools as the long-form benchmarks. No short-form data is used during RL training, making these evaluations fully out-of-domain.

Note on Benchmarks. Unlike Dr. Tulu, we do not include the SQA-v2 (ScholarQA) benchmark (Asai et al., 2024; Bragg et al., 2025) in our evaluation. SQA-v2 is highly citation-centric: it primarily measures whether a model can produce precise inline citations to specific academic papers, which corresponds more to the verifiable citation-grounding component of deep research than to the open-ended answer-quality setting studied here. Our RL objective intentionally uses only rubric-based judge rewards and does not include citation-specific rewards, so SQA-v2 would test a capability that our method is not designed to directly optimize. This does not mean that citation quality is absent from our evaluation: several of the remaining long-form benchmarks, such as ResearchQA and ResearchRubrics, still include citation-, grounding-, or evidence-use-related rubric items. However, these benchmarks evaluate citation use as one part of broader long-form answer quality rather than making precise academic citation the dominant objective. In addition, during SFT data generation, Gemini-3.1-Pro produces few and often unreliable academic citations even under strong prompting and rejection sampling, making it a weak teacher for this specific skill. We therefore leave citation-specific RL and citation-heavy academic evaluation as future work, and focus here on improving semantic quality, coverage, reasoning, and synthesis beyond verifiable rewards.

Additional notes. For a fair comparison with baselines, the main results in Table 1, the RL ablations in Figure 5, and the short-form transfer results in Table 2 are evaluated with direct zero-shot prompts: we do not append rubric-bank entries, previous reflections, or other experience examples at test time. The agent still follows the same benchmark protocol and uses the same external search tools when the task requires tool use. Thus, the reported gains are not simply due to giving RUBRICEM extra reflection context during evaluation. Although the task policy uses reflection context during RL training, the main evaluations test the resulting task policy directly, suggesting that reflection-conditioned training likely results in higher-quality rollouts. We separately study inference-time experience reuse in Figure 6(c), where rubric-bank entries are explicitly injected during evaluation.

F.3. Infrastructure

Our training and evaluation infrastructure builds on the DR Tulu codebase (Shao et al., 2025), which provides the Ray-based distributed RL training loop, vLLM integration for rollout generation, asynchronous tool calling, sample packing, and one-step asynchronous training (Noukhovitch et al., 2024). We use GRPO (Shao et al., 2024) with token-level loss aggregation, tool output token masking (Jin et al., 2025), and a small KL penalty (0.001) for stability. Full hyperparameters are provided in the RL training table above.

We extend the base infrastructure in several ways to support RUBRICEM:

- **Stagewise scoring pipeline:** An asynchronous stagewise judge scoring module runs Gemini API calls in parallel with the training loop. Rubric generation (32 calls per step) and trajectory scoring (256 calls per step) are parallelized using `asyncio.gather` with exponential-backoff retries (see Appendix C).
- **Meta-policy training pipeline:** Reflection generation and meta-policy training are integrated via a three-thread architecture with one-step deferred reflection training, where reflection samples from step N are trained in Phase A of step $N+1$ while the inference engine generates new rollouts. This adds effectively no extra wall-clock overhead (see Appendix D).
- **Rubric bank module:** A thread-safe in-memory store with a FAISS index over query embeddings (Qwen3-Embedding-0.6B on CPU). Bank operations (retrieval, prompt injection, and background reflection generation) are integrated into the data preparation thread with one-step deferred execution (see Appendix D).
- **Checkpoint management:** In addition to standard model and optimizer checkpoints (via DeepSpeed), we save the rubric buffer state, rubric bank contents, and dataloader state at regular

intervals to enable seamless training resumption with consistent memory and data ordering.

For evaluation, we use vLLM to serve the model checkpoint on a single GPU and run the agent pipeline with real-time tool execution via the MCP backend. Each benchmark evaluation varies around 2–18 hours depending on the number of examples, the complexity of the query, judging latency, and the average number of tool calls per trajectory.

Search tools. We expose two search tools to the agent. The primary tool is `google_search`, implemented through Gemin-3-Flash (as the browsing model) with Google Search grounding enabled¹. It returns AI-synthesized summaries with grounding snippets, and is mainly used for general web search, fact-seeking questions, and broad real-world information gathering. The second tool is `snippet_search`, implemented through the Semantic Scholar API². It retrieves text snippets from academic papers and is mainly used when the query requires scholarly evidence, paper-level context, or scientific literature support. Both tools are called through the same MCP backend, and their outputs are wrapped in `<tool_output>` blocks before being returned to the agent.

F.4. Baselines

We compare against baselines from three categories, following the selection of Dr. Tulu (Shao et al., 2025) and supplementing with additional recent models. Scores for existing baselines are taken from the Dr. Tulu paper where available; for models not covered, we run the same evaluation pipeline.

- **Closed deep research models:** Proprietary systems with full deep research capabilities, including OpenAI Deep Research (OpenAI, 2025), Gemini Deep Research (Google DeepMind, 2025), Perplexity Deep Research (Perplexity AI, 2025), Claude Sonnet Search, Gemini 3.1 Pro + Search, and GPT-5 + Search.
- **Fixed pipeline deep research models:** Models that use a fixed multi-stage pipeline (search → synthesis → report) without end-to-end RL training, including WebThinker (Li et al., 2025e) and Ai2 ScholarQA (Singh et al., 2025).
- **Open deep research models:** Open-weight models trained end-to-end for deep research, including Search-R1-7B (Jin et al., 2025), WebExplorer-8B (Liu et al., 2025), Tongyi DeepResearch-30B-A3B (Tongyi DeepResearch Team et al., 2025), and DR Tulu-8B (Shao et al., 2025) (both SFT and RL checkpoints).

DR Tulu is the most direct comparison, since we use the same training data and operate in the same open deep research setting. Compared to Dr. Tulu, RUBRICEM uses a different search tool (Google Search with Gemini grounding vs. Serper API) and a different SFT teacher (Gemini-3.1-Pro vs. GPT-5). Our search tool provides richer AI-synthesized summaries but fewer raw webpage URLs; their teacher is stronger but their search tool returns shorter snippets. We discuss this trade-off in the main text and control for it through ablation studies that isolate the contribution of our training recipe from search tool effects.

G. Algorithm

The algorithm of the paper is described in full in Algorithm 1.

¹<https://ai.google.dev/gemini-api/docs/google-search>

²<https://api.semanticscholar.org/api-docs>

H. Limitations and Discussions

Limitations. Our experiments involve long-horizon agentic RL with search tool calls and external LLM judging, which makes the training loop more sensitive to infrastructure instability than standard offline RL or supervised fine-tuning. During training, we occasionally observed API delays and inconsistent network connections, so wall-clock execution and some rollout-judge latencies were not perfectly controlled across all RL steps. In the main large-scale RL run, the training server also had to be shut down and restarted several times. Although we restored training from checkpoints, such interruptions can introduce additional staleness in the asynchronous reflection branch, rubric bank, and judge-feedback pipeline beyond the intended one-step lag. We therefore view our reported results as reflecting a realistic but not fully ideal infrastructure setting; in principle, a more stable, uninterrupted training environment could reduce stale feedback and further improve the efficiency of our method. Another limitation is that we use Gemini Flash as a cost-effective judge for rubric generation, stagewise scoring, and reflection evaluation. A stronger or more specialized judge could likely provide more accurate stage-level credit and higher-quality reflection rewards, especially for subtle long-form research tasks, but would also increase cost and latency. Exploring the scaling behavior of RUBRICEM with stronger judges, multiple judges, or calibrated judge ensembles is an important direction for future work.

Discussion and broader impact. More broadly, our results suggest that LLM-generated rubrics should be treated not only as evaluation artifacts, but as a general interface for structuring agent behavior, assigning semantic credit, and accumulating reusable experience. They also point to a training-time view of Meta-RL for language agents: rather than using reflection, memory, or self-improvement only as inference-time prompting mechanisms, one can train a meta-policy during RL so that judged experience directly shapes both the agent’s parameters and its reusable textual memory. While we study open-ended deep research beyond verifiable rewards, the same recipe may transfer to other domains where quality is multidimensional and hard to reduce to exact-answer correctness, such as writing assistance, data analysis, scientific review, tutoring, and complex tool-use workflows. At the same time, rubric-guided meta-policies inherit the risks of their judges and rubrics: poorly specified criteria can reinforce shallow preferences, biased standards, or overconfident synthesis, and reusable memories may propagate these errors across tasks. Future work should study more robust rubric generation, stronger or ensemble judges, human-auditable rubric banks, uncertainty-aware reflection training, and safety-aware criteria for domains where agentic research outputs may influence real decisions.

Algorithm 1 RUBRICEM: Reinforcement Learning with Stage-Structured GRPO and Reflection Meta-Policy Training

Require: Structured SFT policy π_θ , reference policy π_{ref} , training queries $\mathcal{D}$, tool environment $\mathcal{T}$, judge $\mathcal{J}$, stages $\mathcal{S} = \{\text{PLAN}, \text{RESEARCH}, \text{REVIEW}, \text{ANSWER}\}$, stage matrix Λ , rollout size n , reflection samples m , active rubric caps $C_{1:K}$

- 1: Initialize active judge rubric buffers $\mathcal{B}_{q,k}^{\text{act}} \leftarrow \emptyset$ for each query q and stage k
- 2: Initialize persistent judge rubrics $\mathcal{B}_{q,k}^{\text{pers}}$ when available, and agent rubric bank $\mathcal{M} \leftarrow \emptyset$
- 3: Initialize deferred reflection batch $\mathcal{P}_0^{\text{ref}} \leftarrow \emptyset$
- 4: **for** RL step $t = 1, \dots, T$ **do**
- 5: $(Q_t, \mu_t) \leftarrow \text{WINDOWEDBATCH}(\mathcal{D}, t)$ ▷ $\mu_t \in \{\text{CROSS}, \text{WITHIN}\}$
- 6: **for all** $q \in Q_t$ **do**
- 7: $E_q \leftarrow \text{RETRIEVE}(\mathcal{M}, q, \mu_t)$ ▷ retrieve reusable experience
- 8: **end for**
- 9: ▷ Deferred reflection meta-policy update
- 10: **if** $\mathcal{P}_{t-1}^{\text{ref}} \neq \emptyset$ **then**
- 11: Update shared π_θ on reflection tokens using GRPO rewards in $\mathcal{P}_{t-1}^{\text{ref}}$
- 12: **end if**
- 13: ▷ Stage-structured task-policy rollout and judging
- 14: Initialize task batch $\mathcal{P}_t^{\text{task}} \leftarrow \emptyset$
- 15: **for all** $q \in Q_t$ in parallel **do**
- 16: Sample tool-augmented rollouts $\{\tau_i\}_{i=1}^n \sim \pi_\theta(\cdot \mid q, E_q; \mathcal{T})$ under scaffold $\mathcal{S}$ ▷ task policy rollouts
- 17: $\Delta \mathcal{B}_{q,1:K} \leftarrow \text{GENERATESTAGERUBRICS}(\mathcal{J}, q, \{\tau_i\}_{i=1}^n, \mathcal{B}_{q,1:K}^{\text{act}})$ ▷ contrast rollouts
- 18: **for** $k = 1, \dots, K$ **do**
- 19: $\mathcal{B}_{q,k}^{\text{act}} \leftarrow \mathcal{B}_{q,k}^{\text{act}} \cup \Delta \mathcal{B}_{q,k}$
- 20: $\mathcal{B}_{q,k} \leftarrow \mathcal{B}_{q,k}^{\text{pers}} \cup \mathcal{B}_{q,k}^{\text{act}}$
- 21: **end for**
- 22: $R_{i,k} \leftarrow \text{SCORESTAGE}(\mathcal{J}, q, \tau_i, \mathcal{B}_{q,k})$ for all $i = 1, \dots, n$ and $k = 1, \dots, K$
- 23: **for** $k = 1, \dots, K$ **do**
- 24: $G_{i,k}^\Lambda \leftarrow \sum_{j=k}^K \lambda_{k,j} R_{i,j}$ for all i
- 25: $A_{i,k} \leftarrow \frac{G_{i,k}^\Lambda - \text{mean}_{i'}(G_{i',k}^\Lambda)}{\text{std}_{i'}(G_{i',k}^\Lambda) + \epsilon}$ for all i
- 26: $\mathcal{B}_{q,k}^{\text{act}} \leftarrow \text{PRUNEBYDISCRIMINATION}(\mathcal{B}_{q,k}^{\text{act}}, \{R_{i,k}\}_{i=1}^n, C_k)$ ▷ evolve stagewise rubric buffer
- 27: **end for**
- 28: $\mathcal{P}_t^{\text{task}} \leftarrow \mathcal{P}_t^{\text{task}} \cup \{(q, \tau_i, B_{i,1:K}, A_{i,1:K})\}_{i=1}^n$
- 29: Launch $\text{PREPAREREFLECTIONBATCH}(q, \{\tau_i, R_{i,1:K}\}_{i=1}^n, \mathcal{B}_{q,1:K}, \mathcal{M}, m)$ asynchronously
- 30: **end for**
- 31: ▷ Task-policy update
- 32: Update π_θ on $\mathcal{P}_t^{\text{task}}$ with SS-GRPO using stage advantages $A_{i,k}$ and KL to π_{ref} by Eq. 1
- 33: $\mathcal{P}_t^{\text{ref}} \leftarrow$ completed asynchronous reflection batches from step t
- 34: **end for**
- 35: **return** trained policy π_θ and agent rubric bank $\mathcal{M}$

function $\text{PREPAREREFLECTIONBATCH}(q, \{\tau_i, R_{i,1:K}\}_{i=1}^n, \mathcal{B}_{q,1:K}, \mathcal{M}, m)$

- 36: Sample one trajectory τ_s uniformly from $\{\tau_i\}_{i=1}^n$ ▷ fixed reflection context
- 37: Generate reflection candidates $\{s_\ell\}_{\ell=1}^m \sim \pi_\theta^{\text{ref}}(\cdot \mid q, \tau_s)$
- 38: **for** $\ell = 1, \dots, m$ **do**
- 39: $(u_\ell^{\text{within}}, u_\ell^{\text{cross}}) \leftarrow \text{JUDGEREFLECTION}(\mathcal{J}, q, \tau_s, s_\ell, R_{s,1:K}, \mathcal{B}_{q,1:K})$
- 40: $u_\ell \leftarrow \frac{1}{2}(u_\ell^{\text{within}} + u_\ell^{\text{cross}})$
- 41: **end for**
- 42: $s^* \leftarrow \arg \max_\ell u_\ell$ among valid reflection candidates
- 43: $\mathcal{M} \leftarrow \text{WRITEBANK}(\mathcal{M}, q, s^*)$ ▷ store only the best accepted reflection
- 44: **return** $\{(q, \tau_s, s_\ell, u_\ell)\}_{\ell=1}^m$
- 45: **end function**
